A Reliability-Aware Comparison of Aggregation Methods for Catalog-Recorded Descriptor Prediction in Two-Molecule Fragrance Blends

Sofia Gabrielle I. Belarmino
Bachelor of Science in Computer Science

Lester Liby V. Ojeñar
Bachelor of Science in Computer Science

Senior Thesis submitted to the faculty of the
Department of Computer Science
College of Computer Studies, Ateneo de Naga University
in partial fulfillment of the requirements for their respective
Bachelor of Science degrees

Project Advisor: Ian Peter L. Lastimosa
Jelly P. Aureus, MS
Jerome G. Flores
Jasper V. Gomez

September 2026
Naga City, Philippines

Keywords: graph neural networks, multi-label classification, set learning

Copyright 2026, Sofia Gabrielle I. Belarmino and Lester Liby V. Ojeñar

The Senior Project entitled

A Reliability-Aware Comparison of Aggregation Methods for Catalog-Recorded Descriptor Prediction in Two-Molecule Fragrance Blends

developed by

Sofia Gabrielle I. Belarmino

Bachelor of Science in Computer Science

Lester Liby V. Ojeñar

Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees has been rigorously examined and recommended for approval and acceptance.

Jelly P. Aureus, MS
Panel Member
Date signed: _____

Jerome G. Flores
Panel Member
Date signed: _____

Jasper V. Gomez
Panel Member
Date signed: _____

Ian Peter L. Lastimosa
Project Advisor
Date signed: _____

The Senior Project entitled

A Reliability-Aware Comparison of Aggregation Methods for Catalog-Recorded Descriptor Prediction in Two-Molecule Fragrance Blends

developed by

Sofia Gabrielle I. Belarmino

Bachelor of Science in Computer Science

Lester Liby V. Ojeñar

Bachelor of Science in Computer Science

and submitted in partial fulfillment of the requirements of their respective Bachelor of Science degrees is hereby approved and accepted by the Department of Computer Science, College of Computer Studies, Ateneo de Naga University.

Jelly P. Aureus, MS

Chair, Department of Computer Science

Date signed: _____

Marianne A. Tolentino, MS

Dean, College of Computer Studies

Date signed: _____

Declaration of Original Work

We declare that the Senior Project entitled

A Reliability-Aware Comparison of Aggregation Methods for Catalog-Recorded Descriptor Prediction in Two-Molecule Fragrance Blends

which we submitted to the faculty of the

Department of Computer Science, Ateneo de Naga University

is our own work. To the best of our knowledge, it does not contain materials published or written by another person, except where due citation and acknowledgement is made in our senior project documentation. The contributions of other people whom we worked with to complete this senior project are explicitly cited and acknowledged in our senior project documentation.

We also declare that the intellectual content of this senior project is the product of our own work. We conceptualized, designed, encoded, and debugged the source code of the core programs in our senior project. The source code of third party APIs and library functions used in our program are explicitly cited and acknowledged in our senior project documentation. Also duly acknowledged are the assistance of others in minor details of editing and reproduction of the documentation.

In our honor, we declare that we did not pass off as our own the work done by another person. We are the only persons who encoded the source code of our software. We understand that we may get a failing mark if the source code of our program is in fact the work of another person.

Sofia Gabrielle I. Belarmino
4 - Bachelor of Science in Computer Science

Lester Liby V. Ojeñar
4 - Bachelor of Science in Computer Science

This declaration is witnessed by:

Ian Peter L. Lastimosa
Project Advisor

A Reliability-Aware Comparison of Aggregation Methods for Catalog-Recorded Descriptor Prediction in Two-Molecule Fragrance Blends

by

Sofia Gabrielle I. Belarmino and Lester Liby V. Ojeñar

Project Advisor: Ian Peter L. Lastimosa

Department of Computer Science

ABSTRACT

Combining two molecular representations is an unordered-pair learning problem: exchanging the two inputs should not change the prediction, while the model may still need to preserve relationships between them. This study will compare mean pooling, order-averaged concatenation, and Set Transformer aggregation for multi-label prediction of catalog-recorded descriptors of two-molecule blends. The three variants will use a shared molecular graph encoder and matched training conditions. Order-averaged concatenation will be treated as an adapted baseline based on a published pairwise concatenation design, not as a full reproduction of that model.

The catalog descriptors will be treated as proxy targets rather than controlled sensory-panel measurements. Reliability-aware eligibility masking and component-disjoint partitioning will be used as evaluation controls: the first will account for uncertain catalog annotations, while the second will reduce repeated molecular components across data partitions. The models will be compared on reliability-screened test cells using macro F1 as the primary metric and micro F1, PR-AUC, and Hamming loss as supporting metrics. Additional analyses will examine catalog-label patterns and how the relative performance of the three methods changes when training-label reliability is reduced, including under controlled label corruption. Numerical results will be reported only after the finalized evaluation pipeline has been run.

TABLE OF CONTENTS

1 Introduction **1**

1.1 Project Context 2

1.2 Purpose and Description 3

1.3 Objectives 5

1.4 Scope and Limitations 5

1.5 Definition of Terms 7

2 Review of Related Literature **10**

2.1 Computational Olfaction and Single-Molecule Prediction 10

2.1.1 History of QSOR Modeling 10

2.1.2 The Principal Odor Map 11

2.1.3 Limitations of Single-Molecule Models 12

2.2 Psychophysics of Odor Mixtures 12

2.2.1 Catalog-Label Patterns in Blends 13

2.2.2 Evidence That Averaging May Be Sufficient 13

2.3 Machine Learning on Sets and Chemical Mixtures 14

2.3.1 Permutation Invariance and Deep Sets 14

2.3.2 Set Transformer (SAB and PMA) 15

2.3.3 Three Ways to Combine Two Vectors 15

2.3.4 Precedents in Mixture Modeling 16

2.4 Olfactory Datasets and Label Reliability 17

2.4.1 Reference Catalogs 17

2.4.2 The Blended-Pair Dataset . . . 17
2.4.3 Human-Rated Datasets . . . 18
2.4.4 Absence of a Standard Vocabulary . . . 18
2.4.5 Noisy and Imbalanced Labels in Multi-Label Learning . . . 18
2.5 Out-of-Distribution Splitting Strategies . . . 19
2.6 Evaluating and Comparing Models Under Label Sparsity . . . 20
2.7 Synthesis and Research Gap . . . 21
2.7.1 Synthesis of Literature . . . 21
2.7.2 Statement of the Research Gap . . . 22
2.7.3 Positioning of the Present Study . . . 22

3 Theoretical and Conceptual Framework 23
3.1 Theoretical Framework . . . 23
3.1.1 Representing Molecules as Graphs . . . 23
3.1.2 Graph Neural Networks and Message Passing . . . 24
3.1.3 Graph Attention Encoder . . . 25
3.1.4 Combining an Unordered Pair . . . 27
3.1.5 Representation Trade-offs . . . 29
3.1.6 Sparse Labels and Label Reliability . . . 30
3.2 Conceptual Framework . . . 32
3.2.1 Input Phase . . . 32
3.2.2 Process Phase 1: Data Preparation and Graph Carving . . . 33
3.2.3 Process Phase 2: Encoding and Pair Representation . . . 34
3.2.4 Process Phase 3: Classification and Optimization . . . 34
3.2.5 Output Phase . . . 35
3.3 Technical Background . . . 35
3.3.1 Cheminformatics Standard (RDKit): SMILES Parsing and Sanitization . . . 35
3.3.2 Deep Learning Stack: Sparse Graph Convolutions . . . 36
3.3.3 Multi-Label Formulation: Sigmoids and Sparsity Mitigation . . . 36
3.3.4 Evaluation Metrics for Sparse Multi-Label Outputs . . . 38

4 Methodology **39**

4.1 Research Design . . . 39

4.2 Data Acquisition and Preparation . . . 41

4.2.1 Data Sources and Provenance . . . 41

4.2.2 Catalog Targets and Their Limits . . . 43

4.2.3 Target Vocabulary . . . 43

4.3 Label Reliability Mask . . . 43

4.3.1 Consistency Evidence . . . 44

4.3.2 Eligibility Mask . . . 44

4.3.3 Experimental Label Conditions . . . 45

4.4 Model Architecture and Training . . . 45

4.4.1 Shared Graph Encoder . . . 46

4.4.2 Aggregation Variants . . . 46

4.4.3 Output Head and Loss . . . 47

4.5 Component-Disjoint Partitioning and Graph Carving . . . 48

4.5.1 Scaffold Check . . . 48

4.5.2 Cluster-Based Partitions . . . 49

4.5.3 The Graph-Carving Protocol . . . 50

4.5.4 Training-Only Vocabulary Filter . . . 50

4.6 Experiments . . . 50

4.6.1 Experiments A and B: Main Comparison (RQ1) . . . 51

4.6.2 Experiment C: Controlled Label Corruption (RQ3) . . . 51

4.6.3 Blend-Type Analysis (RQ2) . . . 51

4.7 Evaluation Metrics . . . 52

4.7.1 Multi-Label Metrics . . . 52

4.7.2 Masked Metrics . . . 52

4.8 Statistical Comparison . . . 53

A Chemical Feature Vectors **55**

A.1 Symmetry Collapse: Representation-Theoretic Derivation . . . 57

A.2 Dataset Profiles and Canonical Vocabulary . . . 58

A.2.1 Single-Molecule Harmonized Dataset Sample . . . 58

A.2.2 Pairwise Blend Mixture Sample 58
A.2.3 Standardized 104-Odor Descriptor Vocabulary Gate 58
A.3 Project Timeline and Gantt Charts 59
A.3.1 Topic to Graded Defense 1 (August – November 2026) 59
A.3.2 Post-Graded Defense 1 to Final Defense (November 18, 2026 – February 2027) 61

B Prototype Application **63**
B.1 Purpose and Status 63
B.2 Main Features 63
B.2.1 Home Page 63
B.2.2 Blend Studio 64
B.2.3 Results by Aggregation Method 64
B.2.4 About the Study and Molecule Dictionary 66
B.3 Basic Usage Flow 66
B.4 Limitations of the Prototype 67

LIST OF FIGURES

3.1 Input-Process-Output (IPO) Model. 32
4.1 Overview of the implemented pipeline. The three aggregation methods receive the same frozen embeddings and the same evaluation protocol, and each has its own separately trained prediction head. 40
A.1 Topic to Graded Defense 1 Gantt Chart (August – November 2026). 60
A.2 Post-Graded Defense 1 to Final Defense Gantt Chart (November 18, 2026 – February 2027). 62
B.1 Home page of the prototype application. 64
B.2 Blend Studio with Molecule A and Molecule B selected. 65
B.3 Results card showing the three aggregation method tabs. 65
B.4 About page of the prototype application. 66
B.5 Molecule Dictionary with partition filters. 67

LIST OF TABLES

3.1 Aggregation methods for an unordered pair of molecule embeddings. 27
4.1 Varied factors and matched controls. 41
4.2 Data sources. 42
4.3 Cell-level rules for the eligibility mask. 44
4.4 The three aggregation variants. 47
4.5 Blend-level training settings for the three variants. 48
A.1 Specification of the 46-Dimensional Atom (Node) Feature Vector (X_i). 56
A.2 Specification of the 10-Dimensional Bond (Edge) Feature Vector (E_{ij}). 56
A.3 Representative sample of harmonized single odorants. 58
A.4 Sample pairwise odor mixtures from the Sisson et al. benchmark with catalog-recorded descriptors. 58
A.5 The 104 standardized odor descriptors (alphabetical). 59

Chapter 1

Introduction

Combining two graph-derived vectors is an unordered-pair learning problem. Each molecule can be encoded as a graph and represented by a learned vector, but a blend should produce the same prediction whether the inputs are presented as (A, B) or (B, A). The combination method must therefore preserve input-order invariance while retaining information about the particular pair being modeled. Mean pooling is a simple order-independent baseline, but averaging can make different pairs look identical to the model and discard information that identifies the pair. More expressive aggregation may preserve those distinctions, but it also adds parameters and computation. Whether that added complexity produces a useful improvement is an empirical question, not an assumption of this study [37].

Fragrance-blend prediction provides the application setting for this pairwise learning problem. Graph Neural Networks (GNNs) have been used to learn molecular representations directly from structure, including in single-molecule odor prediction [21, 45]. Extending a single-molecule encoder to a two-molecule input requires a separate aggregation design. This study compares mean pooling, order-averaged concatenation, and Set Transformer aggregation for predicting catalog-recorded scent labels of binary fragrance blends [22, 33].

The blend scent labels used for training and evaluation are catalog records, not direct measurements of human perception. Public blend datasets such as the one compiled by Sisson et al. [33] are based on reference-catalog recommendations rather than a controlled sensory panel. The study therefore measures label consistency with two checks before comparing the aggregation methods:

two-page scent-label intersection (whether the catalog pages of both component molecules list the same scent label for the blend) and cross-catalog agreement (whether two independent catalogs assign the same scent labels to a molecule). It reports performance under declared reliability conditions rather than treating every catalog label as unquestioned ground truth.

1.1 Project Context

The fragrance domain supplies a useful application for studying this representation problem. Existing molecular GNNs can encode individual structures, but a pairwise blend task requires a second design decision: how to combine two molecule embeddings into one order-independent representation before multi-label classification [21, 33]. The unresolved issue addressed here is therefore not computational olfaction in general. It is the controlled comparison of alternative pair-combination methods under the same encoder, training conditions, and evaluation protocol.

The data structure also creates a partitioning problem. A blend dataset is organized as pair records, and the same molecule can occur in many different pairs. A random split of pair records can therefore place one molecule in both training and test partitions, allowing the model to reuse a component it has already seen. This study uses a component-disjoint split and graph-carving procedure to reduce that overlap by excluding pairs that cross the assigned partitions.

The labels create a second evaluation problem. Each blend may have several scent labels, but only a small fraction of the possible scent-label cells are positive. This sparsity can make certain evaluation metrics appear favorable when a model predicts mostly negative labels. Inconsistent catalog records may also affect the apparent ranking of the aggregation methods. The study addresses these concerns through multi-label metrics suited to sparse outputs and through the reliability audit introduced above, which defines the reliability conditions used in the study.

Fragrance and catalog terminology are retained only to identify the input domain and target scent labels. The study does not attempt to infer biological mechanisms, explain physical mixture chemistry, or model perfume-formulation practice. Its computer-science focus is the representation and evaluation of an unordered pair of molecular graphs.

1.2 Purpose and Description

This study contributes a controlled empirical comparison of mean pooling, order-averaged concatenation, and Set Transformer aggregation under a shared, pretrained molecular-graph encoder. It evaluates whether an attention-based set aggregator provides a measurable benefit over simpler alternatives for predicting catalog-recorded scent labels of two-molecule blends [22, 33].

A single Graph Attention Network v2 (GATv2) encodes each molecular graph into a learned vector [45]. The encoder is pretrained once on single molecules and then frozen, meaning its weights are fixed, so all three methods receive embeddings from the same encoder. The methods differ only in how the two molecule vectors are combined. Mean pooling averages them, order-averaged concatenation joins them side by side in both orders and averages the results, and Set Transformer aggregation lets the two vectors attend to each other before pooling them into one blend representation [22, 47]. Chapter 3 describes each method.

Each aggregation method has its own separately trained classification head, which maps the combined representation to an independent probability (sigmoid output) for each target scent label. The methods are compared using matched data partitions, training conditions, and evaluation metrics.

Significance of the Study

The study is primarily intended to contribute evidence for computer-science research on pairwise representation learning and model evaluation. Its findings may be useful to the following groups:

- **Machine-Learning and Graph-Representation Researchers.** The study provides a controlled comparison of three methods for combining an unordered pair of graph embeddings. It contributes evidence on the performance of attention-based aggregation relative to simpler alternatives, how model comparisons change under uncertain labels, and how component-disjoint splitting can reduce the influence of repeated molecular components across data partitions. Computational olfaction serves as the test domain for this broader representation and evaluation problem.

- **Researchers and Maintainers of Olfactory Reference Catalogs.** The two-page scent-label intersection and cross-catalog agreement analyses provide a way to examine the consistency of catalog-derived labels before they are used for machine learning. These analyses may help identify scent labels that require closer review.

- **Fragrance Researchers and Practitioners.** The results may provide preliminary evidence about the strengths and limits of structure-based prediction for catalog-recorded blend scent labels. They may serve as a reference for future research on computational approaches to fragrance analysis [28].

Research Questions

This study seeks to answer the following questions:

RQ1. How do mean pooling, order-averaged concatenation, and Set Transformer aggregation compare in predicting catalog-recorded scent labels of two-molecule blends under the same training, evaluation, and declared label-reliability conditions?

RQ2. How does the performance of the three aggregation methods vary across groups of blends with different levels of Jaccard overlap between their component scent labels and recorded blend scent labels?

RQ3. How does controlled corruption of training labels at different corruption levels affect the performance of the three aggregation methods?

RQ2 was planned around three groups: additive blends, catalog-label loss (a component label missing from the blend), and catalog-label gain (a blend label that neither component has). Nearly all blends fell into the loss group, so the declared fallback grouping by Jaccard overlap, stated in RQ2, was used instead. For RQ3, the training labels were randomly corrupted at set levels while the validation and test labels were left unchanged.

The study is primarily related to SDG 9 with its work on computational methods and data-quality evaluation. Its findings could also eventually support tools related to responsible material use under SDG 12, but the present study does not measure material savings, reduced laboratory work, or reduced trial-and-error formulation.

1.3 Objectives

General Objective

To compare mean pooling, order-averaged concatenation, and Set Transformer aggregation as methods for combining an unordered pair of molecular graph embeddings for multi-label prediction of catalog-recorded blend scent labels, using a shared pretrained and frozen encoder under a reliability-aware, component-disjoint evaluation protocol.

To achieve the general objective, the following specific objectives must be accomplished:

- To prepare the single-molecule and blend datasets and describe the consistency and sparsity of their catalog-recorded labels.
- To construct fixed component-disjoint data partitions and establish reliability-screened label conditions for model training and evaluation.
- To implement mean pooling, order-averaged concatenation, and Set Transformer aggregation using the same pretrained and frozen GATv2 encoder, with separately trained classification heads and matched training conditions.
- To compare the three aggregation methods using macro F1 as the primary metric, with micro F1, macro average precision, micro average precision, and Hamming loss as supporting metrics.
- To examine how the relative performance of the three methods changes across label-reliability conditions, controlled training-label corruption, and blend groups with different levels of Jaccard overlap between component and blend scent labels.

1.4 Scope and Limitations

This study is limited to pairwise, two-molecule blends and the prediction of catalog-recorded scent labels. Each molecule is represented as a two-dimensional molecular graph and encoded by a shared, pretrained and frozen Graph Attention Network v2 (GATv2) [45]. The study compares mean pooling, order-averaged concatenation, and Set Transformer aggregation as three methods for combining the resulting molecule embeddings. The Set Transformer uses attention to learn statistical relationships between the embeddings [22, 47].

The data are obtained from the GoodScents and Leffingwell single-molecule catalogs and the catalog-derived binary-blend dataset of Sisson et al. [7, 33]. The target scent labels are binary catalog records rather than intensity ratings or measurements from a newly conducted sensory panel. The study does not recruit human assessors and does not claim that its predictions represent what every person would perceive. Concentration, mixing ratio, and changes over time are also outside the scope; each blend is treated as an unweighted pair.

Three scent-label sets are used. The encoder is pretrained on single molecules using 104 standardized reference scent labels. The blend models predict 92 of these labels, the ones with enough training examples. Only the subset of those 92 labels with enough positive and negative examples in the evaluated partition is scored in the macro averages. Chapter 4 gives the exact rules.

Multi-label learning needs both positive and negative targets. A scent label recorded for a blend is treated as positive, and an unlisted scent label is treated as negative under a closed-world assumption. This is an operational definition for model training and evaluation: catalog omission may reflect an unrecorded observation rather than the actual absence of a scent label.

Label reliability is assessed using the two-page scent-label intersection and cross-catalog agreement defined in Section 1.5. These checks measure the consistency of the available records, not their perceptual truth. Cells that the checks cannot support as positive or negative are treated as unknown and masked, meaning they are left out of training and scoring.

The models are evaluated on fixed component-disjoint partitions produced by the clustering and graph-carving procedure described in Chapter 4. Pairs that cross partition boundaries are excluded so that a retained training pair and a retained test pair do not share the same molecule. This design reduces component leakage, but it does not guarantee that the partitions have no structural similarity or that the test molecules represent every possible unseen chemical family [12].

The comparison is limited to the declared model variants, metrics, reliability conditions, and controlled label-corruption experiments. The results therefore show how these methods behave on the selected catalog-derived task and should not be generalized directly to larger mixtures, other scent label vocabularies, or human sensory judgments. Order-averaged concatenation is an adapted baseline based on the pairwise design of Sisson et al.; it is not a full reproduction of that study's model.

This manuscript describes the study at its prototype stage. Data preparation, encoder pretraining, and model training have been completed. The full analysis and discussion of the results are

planned for the next stage of the thesis.

1.5 Definition of Terms

Molecular Graph

A computational representation of a molecule in which atoms are nodes, bonds are edges, and both carry features used by the graph encoder.

Graph Embedding

A fixed-length numerical vector produced by the graph encoder to summarize the learned information from one molecular graph.

Multi-Label Classification

A prediction task in which one blend can be assigned several scent labels at the same time. Each target scent label is predicted separately as present or absent.

Mean Pooling

A combination method that takes the element-wise average of the two molecule embeddings. It is naturally independent of input order, but different pairs can produce the same average.

Order-Averaged Concatenation

A combination method that places the two molecule embeddings side by side, evaluates both input orders, and averages the two resulting logits (the model's raw scores for each scent label, before they are converted to probabilities) before applying a single sigmoid. The final prediction therefore does not depend on which molecule is listed first.

Set Transformer Aggregation

An attention-based method for combining an unordered collection of embeddings. In this study, a Self-Attention Block (SAB) first updates each molecule embedding using the other, so swapping the two inputs only swaps the two updated embeddings. Pooling by Multihead Attention (PMA) then combines them into one blend representation that is the same for either input order.

Permutation or Order Invariance

The requirement that exchanging the positions of the two input molecules must not change the model's final prediction.

Component-Disjoint Split

A data partition in which the same molecule does not occur in both a retained training pair and a retained test pair.

Two-Page Scent-Label Intersection

A scent label that is recorded for a blend on the recommendation pages of both component molecules, so that two pages support it. It is not the overlap between GoodScents and Leffingwell labels for a molecule. It is one of the two checks used for label reliability.

Cross-Catalog Agreement

The agreement between GoodScents and Leffingwell on whether a scent label is recorded for the same molecule. It is measured for each scent label with Cohen's kappa, a standard agreement score that discounts the agreement expected by chance.

Reliability Mask

A cell-level indicator that determines which blend–scent label targets are included in reliability-screened training or evaluation. Eligible cells are included, while cells with unknown evidence are excluded.

Eligible Cell

A single blend–scent label entry that is included in training or scoring because the reliability checks support it: either a recorded positive with enough evidence, or an unlisted scent label treated as a negative.

Unknown Target

A blend–scent label entry whose evidence is too weak to call positive or negative. It is masked, meaning it is left out of training and scoring, and it is not treated as absent.

Jaccard Overlap

A score from 0 to 1 for how much two sets of scent labels share: the number of labels in both sets divided by the number of labels in either set. Here it compares a blend's recorded labels with the

combined labels of its two component molecules.

Support Cutoff

The minimum number of examples a scent label needs before it is scored in the macro averages: at least one eligible positive and one eligible negative in the evaluated partition.

Decision Threshold

The probability above which a model's output for a scent label is counted as present. One threshold is chosen on validation data for each method and label condition.

Evaluation Metrics

Macro F1 gives each supported scent label (one that meets the support cutoff) equal weight when averaging its F1 score. Micro F1 combines all evaluated decisions before computing one F1 score. Average precision (AP) summarizes the model's precision across recall levels, with macro AP averaging across supported labels and micro AP aggregating predictions across labels. Hamming loss reports the fraction of evaluated present-or-absent decisions that are incorrect.

Chapter 2

Review of Related Literature

This chapter reviews papers that help build the idea for this study. It tackles topics such as graph-based scent prediction for single molecules, the processing of unordered molecular pairs, the existing approaches to chemical mixture modeling, the available olfactory datasets, label reliability, and evaluation strategies relevant to the present study.

2.1 Computational Olfaction and Single-Molecule Prediction

This first section explores the evolution of computational olfaction from early QSOR modeling with manual feature engineering to more modern graph-based methods. It reviews how these frameworks were used in predicting scent from a single molecule and the limitations of such when applied to the multi-molecule blend prediction task this thesis aims to achieve.

2.1.1 History of QSOR Modeling

Quantitative Structure-Odor Relationship (QSOR) modeling refers to the prediction of the odors and olfactory properties of molecules through the characterization of their structure [14, 43, 48]. QSOR is especially important in fields such as the fragrance and flavor industries, as it could help reduce the trial-and-error involved in the formulation and development of new compounds computationally, as experimental wet lab methods can be both time-consuming and expensive [19, 29].

Early approaches in QSOR involved the manual creation of numerical features that describe

and quantify the properties of a molecule [3, 14, 49]. These included molecular fingerprints, which encode the presence or absence of substructures in a binary vector, and physicochemical descriptors computed with dedicated software [3, 10, 31, 42].

Many studies pair molecular descriptors or fingerprints with classical machine learning models, such as random forest, XGBoost, and LightGBM, to predict odor [29, 36, 41]. Comparisons across modeling approaches have also shown the potential of different molecular representations. Suh et al. found that Morgan-fingerprint models outperformed descriptor-based models [36], while Achebouche et al. found that graph convolutional representations outperformed traditional molecular descriptors and fingerprints [1].

While effective, these methods require the features to be chosen in advance [4, 21, 49], so the learning model works from a fixed summary of each molecule. Graph neural networks address this by repeatedly updating atom-level features using information from neighboring atoms and their bonds and learning which combinations of features are most useful for the task, although people still choose the input features and the network design [2, 4, 17, 48]. The best-known graph-based odor model is discussed next.

2.1.2 The Principal Odor Map

Addressing the aforementioned limitation is Lee et al.'s Principal Odor Map (POM), which maps a molecule's chemical structure to its smell using Graph Neural Network (GNN) techniques [21]. Rather than evaluating a molecule's scent using a static representation built from predefined hand-crafted descriptors, POM learns directly from the molecule's structure during training. It does this by treating each molecule as a graph, where nodes and edges represent atoms and chemical bonds, respectively; each node and edge is itself a vector, with nodes storing properties such as atomic number, and edges storing bond properties. Through message passing, the GNN dynamically assigns weights to each of the molecule's structural features, producing a numerical representation of the molecule [21].

The setup was as follows: Lee et al. used approximately 5,000 molecules derived from merging data from the GoodScents and Leffingwell fragrance databases to produce a multi-label output of 138 distinct semantic odor descriptors. It was then tested on new molecules against a human benchmark, wherein approximately 15 trained panelists rated molecules using a narrowed-down set of 55 odor descriptors. The model's predictions fell within the range of individual panelists' ratings for every

label except musk, and surpassed the median panelist on 30 out of the 55 labels (55%) [21]. This study differs from theirs because they checked their model against a newly collected human panel, while this study has no panel and evaluates against catalog-recorded labels only.

This shows that a graph model trained on catalog labels can predict panel-rated scent from molecular structure alone at roughly the level of a typical panelist. However, despite this significant advancement, these results were derived exclusively from single, unmixed molecules.

2.1.3 Limitations of Single-Molecule Models

POM shows what a graph encoder can learn for one molecule, but it operates on a single molecule at a time, while odorants rarely appear in isolation [8, 21, 39, 46]. In the real world, scents are rarely composed of single molecules and are instead blends that can contain multiple individual components [16, 46]. A single-graph encoder produces one vector per molecule, but it does not define how the vectors of two molecules should be combined into one prediction. That combining step is the subject of this study.

Lee et al. admit this limitation themselves, noting that while POM uses a coordinate system to categorize scents, it alone cannot determine the behavior and outcomes of different molecules when mixed [21].

2.2 Psychophysics of Odor Mixtures

Because single-molecule models do not specify how two molecules are combined, a method for combining them is needed. This section establishes only that both simple averaging and learned aggregation are plausible candidates for that method. It reviews how catalog blend labels differ from the labels of their parent molecules, and the human-panel evidence that averaging may be sufficient.

Mixture research examines how components are perceived together. Configural perception refers to experiencing a mixture as an odor whole rather than only as its separate components [9]. Controlled human studies also report reduced component odor intensity in particular mixtures [13]. These observations provide context for studying combinations.

2.2.1 Catalog-Label Patterns in Blends

A paper released by Sisson et al. introduces a labeled dataset consisting of molecule pairs and compares each pair's catalog-recorded blend descriptors with the descriptors of its parent molecules using two baselines: union and intersection. The union score assumes that the blend record contains any descriptor listed for either parent (additive labeling). The intersection score assumes that the blend record contains only descriptors shared by both parents (subtractive labeling) [33].

The Jaccard results show 0.12 for union and 0.24 for intersection [33]. These low scores suggest that simple union or intersection rules do not reproduce the catalog-recorded labels of molecule pairs.

Catalog blend labels differ from the labels of the parent molecules in two ways. In catalog-label loss, a scent word recorded for either parent molecule is absent from the blend's labels. In catalog-label gain, the blend's labels contain a scent word that neither parent catalog record has. The results of Sisson et al. show that, on average, 1.2 notes are lost and 0.2 notes are gained per blend. Furthermore, an odds-ratio analysis reveals that certain scents, such as "eggy" or "cabbage", were more likely to be gained, while certain scents like "peach" or "fresh" were more likely to be lost [33]. These catalog differences appear to follow a pattern rather than being random, so it is worth testing whether a model that can learn relationships between the two molecules, such as a Set Transformer, helps.

2.2.2 Evidence That Averaging May Be Sufficient

Not all evidence points to strongly non-additive mixing. Pellegrino et al. [25], in a preprint, had a trained panel rate 432 mixtures using a 51-term list and found that a linear average of the components' ratings predicted mixture profiles about as well as the noise in the panel's own ratings allowed. They argue that apparent emergence often reflects the limits of describing an odor with a few words. Note that this averages the panel's ratings, which is not the same as averaging learned molecule embeddings.

Binary catalog labels, as in Sisson et al. [33], instead record catalog-label loss and gain in many blends. Whether a learned pair-aggregation step helps for binary multi-label prediction is therefore an open question, which this study tests rather than assumes.

2.3 Machine Learning on Sets and Chemical Mixtures

This section is the core of the chapter. It defines permutation invariance, explains how mean pooling, order-averaged concatenation, and Set Transformer aggregation each combine two molecule vectors, and reviews prior work on mixture modeling.

2.3.1 Permutation Invariance and Deep Sets

A combination of two or more molecules could be viewed as a mathematical set. An accurate depiction of a fragrance blend of these molecules requires that they be presented in such a way that their positioning in that list is arbitrary, not sequential [33, 39, 48]. For instance, assume the combination of molecule A and molecule B produces scent C. It should be of no relevance whether it is molecule A + B or molecule B + A; it should always produce scent C. For that reason, it is important for any mathematical model used to predict the resultant smell from the combination of different molecules to follow this principle, which is called permutation invariance. In plain terms, a function is permutation invariant if swapping its inputs does not change its output.

Zaheer et al. [47] established the math of permutation invariance and introduced Deep Sets, a deep learning framework that uses a neural network designed to process datasets in a non-sequential manner for unordered data. This is referenced as the first of two properties they introduced in their study describing permutation invariance, positing that for a function acting upon a set, it must be indifferent to the order in which each element appears, mathematically formulated as $f : 2^X \to Y$ for any permutation, i.e., for any permutation π:

$$f(\{x_1, \ldots, x_M\}) = f(\{x_{\pi(1)}, \ldots, x_{\pi(M)}\}).$$

Their second result explains how to build such a function: transform each element individually (ϕ), aggregate the results, and then transform the aggregate again (ρ) [47].

However, because each element is transformed before pooling and in isolation, any information about how the elements relate to each other may have to be inferred indirectly by later layers from the pooled representation, as noted by Lee et al. [22]. For a pair of molecules, this raises the question of whether letting the two vectors affect each other before pooling helps. This does not mean Deep Sets cannot model these relationships, and averaging two fixed embeddings is not the same as training a full Deep Sets model.

2.3.2 Set Transformer (SAB and PMA)

To bridge this gap, Lee et al. propose the use of a Set Transformer, where instead of processing the elements of a set individually, they apply self-attention mechanisms they call SAB (self-attention block) in the processing of the elements, wherein the different elements are able to learn from each other, such that the extracted features to be used for pooling already contain useful information about the elements' relationship with each other. SAB itself is permutation equivariant, so swapping the order of the input swaps the corresponding outputs. Moreover, they also developed PMA, or Pooling by Multihead Attention, which replaces the second step in the Deep Sets architecture wherein, instead of fixed sum or mean pooling, PMA also employs self-attention mechanisms in the pooling itself, such that it is able to pick out which of the elements bear more weight for the task at hand and gives a permutation-invariant output. Both are candidates for combining the two molecule vectors in this study: SAB lets each vector be updated using the other before pooling, and PMA learns how to weight them. The Set Transformer is therefore treated as a way to model statistical relationships between two learned vectors.

2.3.3 Three Ways to Combine Two Vectors

Three ways of combining the two vectors are compared in this study. Mean pooling averages the two vectors. It is simple and naturally order independent, but it can lose information specific to the pair, because different pairs with the same average produce the same combined representation [37]; this does not mean that it must fail on any particular task. Order-averaged concatenation places the two embeddings side by side in both possible orders, as in the pairwise design of Sisson et al. [33], and averages the two outputs. This adaptation makes the baseline invariant to input order while retaining both embeddings in full. Set Transformer aggregation lets the two vectors affect each other's learned representation before pooling, while remaining independent of order.

The three methods also differ in cost. Self-attention blocks in the Set Transformer scale quadratically with the number of elements in a set [22]; for a set of two molecules this cost is small, but attention blocks still add trainable parameters and computation compared with fixed pooling. A fair comparison therefore reports parameter counts and computation cost alongside accuracy, so that any gain can be weighed against the extra complexity.

2.3.4 Precedents in Mixture Modeling

Zaheer et al. [47] provided the groundwork for permutation invariance in sets, while Lee et al. [22] extended it with self-attention, a general set method that later studies applied to molecular mixtures.

Qin et al. [26] built SolvGNN to predict activity coefficients of binary and ternary chemical mixtures using two levels of graph processing: graph convolutions inside each individual molecule, followed by a molecule-level graph whose nodes are the molecules themselves. They argue that modeling the mixture explicitly at this second level matters in cases where naive pooling falls short [26].

Similarly, Sanchez Medina et al. [31] introduce GH-GNN, which also predicts activity coefficients of chemical mixtures. It uses a single shared molecular encoder whose pooled output is passed to a mixture-level graph. This resembles the structure of the present study, a shared single-molecule encoder followed by a mixture-level step, but applied here to odorant pairs rather than to solvent and solute.

Lastly, Tom et al. [39] proposed POMMix, a more relevant precedent for our topic, as it is in the field of olfactory perception, aiming to predict the similarity of the scent of two mixtures. It employs a module they call CHEMIX, which uses a self-attention mechanism, doing set-pooling to produce a permutation-invariant mixture embedding. POMMix differs from this study in its prediction task: it predicts the similarity between two mixtures, whereas this study predicts the scent labels of a two-molecule blend.

These three studies tackle different prediction problems. Evidence from solvent mixtures in particular does not transfer directly to scent labels. They motivate evaluating learned aggregation for odor blends; they do not establish that it outperforms simple pooling for the binary scent labels used here.

Hierarchical GNNs have also been used for property prediction within a single molecule, moving from atoms to fragments, as in HiGNN and HimGNN [17, 49]. POMMix, SolvGNN and GH-GNN, in contrast, operate across molecules. The present study follows the second kind: a shared single-molecule encoder followed by a pair-aggregation step for the two molecule embeddings.

2.4 Olfactory Datasets and Label Reliability

This section distinguishes the single-molecule catalog labels from the pairwise blend labels, describes what each can and cannot support, and explains why label inconsistency, label noise, and label sparsity matter for evaluation.

2.4.1 Reference Catalogs

Public fragrance platforms and reference catalogs contain different types of scent information. Community platforms such as Fragrantica and Basenotes primarily describe finished commercial perfumes through user-submitted reviews and ratings, which makes them unsuitable for learning how individual chemicals relate to scent [7, 42]. Because of this, researchers use catalogs such as GoodScents and Leffingwell for reference, as they record descriptors for individual aroma chemicals and have been used to train prediction models [21, 33].

2.4.2 The Blended-Pair Dataset

Sisson et al. [33] compiled blend labels from the lists of suggested blending partners on GoodScents, which group each partner under an odor heading. Their crawl was able to produce more than 160,000 pairs and, after standardization and filtering, resulted in up to 74 odor labels. Each molecule's page was crawled separately, so a pair can be listed from both molecules' pages. The dataset has no concentration information, and its labels are catalog suggestions and not measurements from an actual sensory panel. In this study they are therefore treated as catalog-recorded blend labels and used as proxy targets.

Note as well that the pair dataset used in this study is not assumed to be identical to the one used by Sisson et al., and that the pair scent labels are separate from the single-molecule labels in GoodScents and Leffingwell, which describe only single molecules.

Furthermore, a scent label may appear from one direction only (e.g., molecule A lists B as a potential blending partner for scent X, but B does not list A). Requiring both molecule pages to list each other for that scent label is one way to reduce reliance on a single recommendation. This study calls this blend recommendation evidence, which is defined in Chapter 4. The trade-off is that it reduces the number of eligible labels, and it cannot show that a label is perceptually correct.

2.4.3 Human-Rated Datasets

Although expert-compiled catalogs provide human-assigned scent descriptions, they are also the sources of the labels used to train the models in this study. They therefore cannot independently establish whether the predictions agree with human perception. Human-rated datasets provide a separate basis for evaluation, but existing resources cover only parts of the task addressed in this study. For single molecules, Bierling et al. [6] collected yes-or-no ratings for 16 scent words across 74 odors from approximately 1,200 laypeople. For mixtures, Pellegrino et al. [25] collected trained-panel ratings using a 51-term descriptor list. However, to our knowledge, no publicly available dataset provides human-rated scent descriptors specifically for two-molecule blends as of November 2026.

2.4.4 Absence of a Standard Vocabulary

No standard set of odor descriptors exists. Catalogs and studies use vocabularies of different sizes, and many words overlap in meaning or differ only in spelling, such as “acid” and “acidic” [33]. Sajan et al. [30] grouped raw odor words from the Pyrfume repository into 16 broader scent families, but this study does not use those families because their words are not directly equivalent to the descriptors in the present data. The models therefore predict only the standardized descriptors defined for this dataset. How the vocabulary is standardized and filtered, and how unlisted descriptors are treated, is described in Chapter 4.

2.4.5 Noisy and Imbalanced Labels in Multi-Label Learning

Inconsistent labels act as noise in both the training targets and the test targets. Noise in the training targets can change what a model learns, because deep networks can memorize noisy labels [34]. Because of that, noise in the test targets can change which model appears best. Noise is also harder to handle with several labels per example, because errors can be correlated across labels; Li et al. [23] model this by estimating a noise transition matrix that uses label correlations. Moreover, missing scent labels are an additional problem because unlisted scent labels may not necessarily be true absences and may simply have been omitted by the catalog. For this reason, this study measures label consistency, evaluates on the most consistent labels, and tests whether the comparison between aggregation methods changes as label reliability decreases.

Multi-label scent data are also sparse as most descriptors are absent for most molecules, and some descriptors are far more common than others. Reviews of imbalanced multi-label classification describe imbalance within labels, among labels, and among label sets [38], and work on graph neural networks reports that the benefit of balancing techniques depends on the dataset and problem type [2]. Because simple accuracy-style scores can look good on sparse labels, evaluation of such data needs metrics that are not dominated by true negatives [27].

2.5 Out-of-Distribution Splitting Strategies

This section provides a review of the standard data-splitting strategy of scaffold splitting, and an explanation of how its application could fall short in sparse chemical spaces. This section then grounds and discusses the clustering-based alternative used in this study.

A common method to partition data for training, testing, and validation in cheminformatics and molecular machine learning is scaffold splitting [15, 35]. In this method, molecules are represented by graphs of their structure, and the core framework of these graphs is referred to as their "scaffold" (Bemis–Murcko scaffolds) [44]. Molecules that share the same scaffold are grouped together, and these groups are then distributed among the training, testing, and validation sets to ensure that the test set contains structurally different molecules from the training set, which is a basic form of out-of-distribution (OOD) evaluation, which as opposed to random splitting, simply distributes molecules randomly across training, validation, and test sets [12, 15, 33, 35].

The caveat with this method is that, when the dataset is sparse, it could potentially collapse toward random splitting. For instance, similarly structured molecules could get fragmented across both the train and test splits, leading to data leakage and inflated test scores as scaffolds in standard benchmarks could be populated by only one or two molecules each, undermining the grouping effect the method depends on [12]. Another problem demonstrated by scaffold splitting is that even molecules that are near-identical (as computed by Tanimoto similarity) [15, 35], even differing by as little as a single atom, can, under this method, end up in opposite splits, as observed by Guo et al. and Steshin et al. [15, 35], who argue that this problem causes models to "memorize" rather than generalize from the data. Because of these problems, scaffold splitting can misleadingly produce scores that are overly optimistic.

Shimakawa et al. [32] propose doing topological pre-clustering by projecting 2048-bit ECFP4

fingerprints onto a 2D map using 2D Uniform Manifold Approximation and Projection (UMAP), then grouping them together using spectral k-means clustering. More generally, leakage-reduced splitting can be formulated as an optimization problem over the data [20], and cluster-based splits have been reported to be among the most challenging for both classical and graph-based models [12].

In plain terms, one molecule can appear in many blend pairs, so splitting pairs at random can place the same molecule in both training and testing. Methods designed for single molecules do not address this, because an individual molecule can be a component of varying other molecules in different blends, meaning a molecule seen during training could reappear as a component in a different, unseen test blend, leaking information about that molecule across the split and inflating the performance metrics. Therefore, Sisson et al. [33] propose "graph carving", using a meta-graph to describe the pairwise blend dataset, where each node represents an individual molecular graph and each edge represents a tested blend combination. Edges connecting a molecule from one partition to a molecule in another are discarded, so that no component is shared between splits. In their original implementation, this partitioning was done by randomly assigning molecules to each side and repeating until every label had at least one training and one test example, since a true scaffold-based split was not compatible with their sparse label distribution.

This study applies the graph-carving protocol on top of a fingerprint-clustering partition (in the style of the UMAP and k-means approach discussed above), rather than random or scaffold-based partitioning. Doing this creates a component-disjoint split that reduces component and structural overlap between training and testing; it does not guarantee that every test molecule is entirely unlike every training molecule.

2.6 Evaluating and Comparing Models Under Label Sparsity

Sparse multi-label targets make some common scores misleading. Precision, recall, and F1 ignore true negatives and so expose a model that predicts no labels, whereas accuracy-style scores such as Hamming loss can look strong for such a model [27]. F1 can be averaged per descriptor (macro), which gives scent labels that are rarely found the same weight as common ones, or pooled across all decisions (micro), which favors common scent labels [27]. Average precision (AP), which is based on the precision–recall curve, summarizes how well a model ranks true scent labels above false ones

across all cutoffs. The cutoff used to turn a probability into a present or absent prediction is also important. A study comparing graph-based and traditional models for multi-label odor prediction found that choosing the right threshold was essential [41]. To ensure fair results, this threshold should be selected without using the test set.

Comparisons between models should also report how large a difference is. Bootstrap confidence intervals do this by showing the difference and its uncertainty together [5]. Recent work warns against naive paired t-tests when resampled test sets are used [27]. Together with the leakage problem above, this means that a fair comparison of aggregation methods needs matched encoders and training conditions, sparsity-aware metrics, validation-only threshold selection, and paired uncertainty estimates over the evaluated blends.

2.7 Synthesis and Research Gap

2.7.1 Synthesis of Literature

Current models in computational olfaction can successfully map single-molecule structures to their appropriate sensory descriptors [21]. However, because these compounds are evaluated in isolation, they do not address how two molecules should be combined, and catalog blend labels differ from the labels of the parent molecules in many blends [33]. Human-panel evidence on mixtures is mixed: continuous ratings are well predicted by a linear average of the components [25], while binary catalog labels record catalog-label loss and gain in many blends [33].

From an architectural standpoint, order-independent set processing is a natural requirement for combining unordered components [47, 22]. Averaging can make different inputs look identical, and a related loss of information has been described for averaging-based readouts within a molecule [37]. For a pair, this is a possible limitation of mean pooling.

Furthermore, standard scaffold splits can behave like a random split and can cause data leakage [12]. Clustering molecules and carving the blend meta-graph along cluster boundaries reduces component and structural overlap between training and testing [32, 33]. Finally, the blend labels in public data come from catalogs rather than sensory panels, and their reliability has rarely been measured in prior blend studies.

In summary: (1) a molecular graph encoder produces one vector per molecule; (2) the unresolved computing problem is how to combine the two vectors without making the prediction depend on

input order; and (3) any such comparison must account for component leakage and uncertain labels.

2.7.2 Statement of the Research Gap

To our knowledge, it remains unclear whether an attention-based set aggregator offers a reliable advantage over simpler aggregation methods when combining two molecular graph embeddings for sparse, catalog-derived multi-label prediction. Evaluating this requires a controlled comparison in which the encoder and training conditions are held constant, input-order effects are addressed, shared molecular components are kept across separate partitions, and label uncertainty is accounted for. This study addresses this gap by comparing the aggregation methods under these controlled conditions.

2.7.3 Positioning of the Present Study

To address this gap, this study compares mean pooling, order-averaged concatenation, and Set Transformer aggregation for combining the two molecule embeddings produced by one shared, frozen GATv2 graph attention encoder, with a separately trained prediction head for each method. The comparison is carried out on a component-disjoint, fingerprint-clustered and graph-carved partition of the data. The blend recommendation evidence check and the controlled label-corruption tests show how far the conclusions depend on the labels. A simpler baseline that matches or outperforms the Set Transformer would be a valid and informative result.

Chapter 3

Theoretical and Conceptual Framework

3.1 Theoretical Framework

This section explains the computational concepts used in the study. It begins with how atoms and bonds are represented as a molecular graph and how a graph neural network produces one numerical vector for each molecule. It then explains how mean pooling, order-averaged concatenation, and Set Transformer aggregation combine two molecule vectors while keeping the final prediction independent of input order. Lastly, it discusses how sparse catalog labels, blend recommendation evidence, and cross-catalog agreement affect learning and evaluation. These concepts provide the basis for the procedures described in Chapter 4.

3.1.1 Representing Molecules as Graphs

Each molecule is represented as a graph in which atoms are nodes and bonds are edges. Each node and edge carries numerical features that become inputs to the graph encoder [45]. This representation preserves which atoms are connected. SMILES strings are used to store and parse the structures before they are converted into graphs [24, 42].

Mathematical Formalism

A molecular graph is written as $G = (V, E)$, where V is the set of non-hydrogen atoms and E is the set of covalent bonds. Hydrogen information is included in the features of the corresponding atoms rather than represented as separate nodes [4].

The connections can be described by an adjacency matrix A, where $A_{ij} = 1$ if atoms i and j share a bond and $A_{ij} = 0$ otherwise [49]. The implementation stores these connections in sparse form, as explained in Section 3.3.2. Each node and edge also has a feature vector:

- The node vector $\mathbf{x}_i \in \mathbb{R}^{d_x}$, with $d_x = 46$, records properties such as element, charge, valence, hybridization, aromaticity, hydrogen count, ring information, and chirality.
- The edge vector $\mathbf{e}_{ij} \in \mathbb{R}^{d_e}$, with $d_e = 10$, records bond properties such as bond type, conjugation, ring membership, and stereochemical configuration.

RDKit supplies the parsed molecular structures and the properties used to construct these vectors. The complete feature definitions are listed in Appendix A.

Atom numbering is arbitrary. Changing the order in which atoms are listed changes their indices, but it does not change the molecule when the same features and connections are preserved. The graph encoder and readout should therefore produce the same molecular embedding regardless of how those atoms are numbered.

3.1.2 Graph Neural Networks and Message Passing

Most neural networks receive inputs in a fixed arrangement. A molecular graph does not have that form: molecules can contain different numbers of atoms, and their important relationships are represented by bonds. A Graph Neural Network (GNN) works with this structure directly. In the graph used here, a *node* represents an atom and its feature vector, while an *edge* represents a bond and its feature vector. The *neighborhood* $\mathcal{N}(i)$ of node v_i is the set of atoms directly connected to it by an edge [4, 49, 41].

The main operation in a GNN is *message passing*. During one layer, each node receives information from its neighborhood, combines those messages, and updates its own representation. For

node v_i at layer t, one general form of this operation is:

$$\begin{aligned} \mathbf{m}_i^{(t)} &= \bigoplus_{j \in \mathcal{N}(i)} M_t\big(\mathbf{h}_i^{(t)}, \mathbf{h}_j^{(t)}, \mathbf{e}_{ij}\big), \\ \mathbf{h}_i^{(t+1)} &= U_t\big(\mathbf{h}_i^{(t)}, \mathbf{m}_i^{(t)}\big). \end{aligned} \tag{3.1}$$

Here, $\mathbf{h}_i^{(t)}$ is the current representation of node i, $\mathbf{e}_{ij}$ describes the bond to neighbor j, and M_t computes the message sent along that bond. The symbol $\bigoplus$ denotes the operation that combines all messages received by node i, while U_t produces its new representation. In ordinary terms, the node looks at itself, looks at its bonded neighbors, and then records an updated summary. Since the messages are combined as a group rather than in a prescribed order, changing the order in which neighboring atoms are stored does not change the information being combined. Repeating the layers allows information to travel from an atom to nearby atoms and then to larger parts of the molecule [17, 49].

After the final message-passing layer, a *graph readout* combines the node representations into one vector for the whole molecule. The encoder used here applies learned attention weights when combining the atom vectors. This atom-to-molecule step is separate from blend-level mean pooling, which averages two completed molecule embeddings. Both molecules use the same encoder and readout [17, 41]. The specific graph-attention encoder is explained in the next subsection.

3.1.3 Graph Attention Encoder

The shared graph encoder converts each molecular graph into one fixed-length vector. It learns patterns in the atom and bond features that support descriptor prediction. Both molecules in a blend pass through the same encoder, so their embeddings are produced using the same learned parameters [21, 45].

The implementation uses three edge-aware `GATv2` layers. GATv2 applies a nonlinear transformation before reducing the node projections to an attention score. Let $\mathbf{h}_i^{(0)} = \mathbf{x}_i$ denote the 46-dimensional input for atom i. The 10-dimensional bond vector $\mathbf{e}_{ij}$ is first projected into a 128-dimensional vector, $\mathbf{r}_{ij} = \mathbf{P}\mathbf{e}_{ij} + \mathbf{b}_e$, which is supplied to each attention layer. Here, $\mathbf{P}$ is a learned projection matrix and $\mathbf{b}_e$ is its learned bias.

For one layer and one attention head, the attention score is:

$$\tilde{e}_{ij} = \mathbf{a}^\top \mathrm{LeakyReLU}(\mathbf{W}_q\mathbf{h}_i + \mathbf{W}_n\mathbf{h}_j + \mathbf{W}_e\mathbf{r}_{ij})\,. \tag{3.2}$$

Here, $\mathbf{W}_q$, $\mathbf{W}_n$, and $\mathbf{W}_e$ are learned projection matrices for the receiving atom, neighboring atom, and projected bond vector, respectively. The learned vector $\mathbf{a}$ reduces the transformed features to one attention score. Projection biases are omitted from this equation for clarity.

Each atom uses information from its bonded neighbors and from itself. Including the atom itself allows its current representation to contribute to the update. This extended neighborhood is written as $\mathcal{N}^+(i) = \mathcal{N}(i) \cup \{i\}$.

Graph attention uses softmax to convert neighborhood scores into normalized weights [45]:

$$\alpha_{ij} = \frac{\exp(\tilde{e}_{ij})}{\sum_{k \in \mathcal{N}^+(i)} \exp(\tilde{e}_{ik})}. \tag{3.3}$$

For each receiving atom, these weights sum to one before attention dropout. A larger weight gives a neighboring representation a greater contribution to the update. The self-connection is computational and does not represent an additional chemical bond. Its edge vector is formed by averaging the incoming projected bond features.

The attention head then calculates a weighted sum:

$$\mathbf{u}_i = \sum_{j \in \mathcal{N}^+(i)} \alpha_{ij} \mathbf{W}_n \mathbf{h}_j. \tag{3.4}$$

Each projected neighboring vector is multiplied by its attention weight, and the results are added together [45]. The vector $\mathbf{u}_i$ represents this combined information before the output bias, normalization, and activation are applied.

The first two layers each use four attention heads. Each head learns its own weighted combination and produces 128 features per atom. The four outputs are concatenated, giving a 512-dimensional atom vector. Layer normalization rescales the features, ELU applies a nonlinear transformation, and dropout randomly masks selected values during pretraining.

A residual connection adds the first layer's output to the second layer's processed output, allowing earlier information to remain available. The third layer uses one attention head to produce a 256-dimensional vector for each atom, followed by layer normalization and ELU. Attention dropout is also applied during encoder pretraining; the softmax equation describes the weights before this dropout.

Attention-based readout combines atom representations through a weighted sum to obtain a molecular representation [45]. In this study, the readout assigns a score to each final atom vector, normalizes the scores across the molecule, and produces one 256-dimensional embedding $\mathbf{z}$. This operation is shared by all three aggregation methods and is separate from blend-level mean pooling.

The encoder is pretrained using single molecules from the training partition, with an internal validation subset and a temporary 104-descriptor prediction head trained using masked binary cross-entropy. After checkpoint selection, the temporary head is removed and the encoder is frozen. During blend training, its parameters remain unchanged and dropout is inactive. Only the aggregation and prediction modules are trained. Chapter 4 describes this procedure.

Attention weights are calculated from learned parameters and the input graph. They describe how information contributes to the model's representation; they do not establish chemical importance, receptor activity, or physical interactions between molecules.

3.1.4 Combining an Unordered Pair

The blend model receives two molecule embeddings, $\mathbf{z}_A$ and $\mathbf{z}_B$, from the shared frozen encoder. Each embedding is a 256-dimensional vector representing one molecule. The order in which these vectors are supplied has no meaning in the task. The model should therefore return the same prediction for (A, B) and (B, A). This requirement is called input-order invariance [33, 47].

The study compares three methods for combining these embeddings: mean pooling, order-averaged concatenation, and Set Transformer aggregation. Each method uses its own prediction head. Unlike the graph readout, which combines atom representations within one molecule, this step combines the representations of two molecules. The following table summarizes how the methods handle input order and their main trade-offs.

Table 3.1: Aggregation methods for an unordered pair of molecule embeddings.

Method	**Pair representation**	**How order is handled**	**Main trade-off**
Mean pooling	Average the two embeddings	Invariant by construction	Can discard pair-specific distinctions
Order-averaged concatenation	Evaluate both orders through one head and average logits	Explicitly symmetrized	Requires two ordered evaluations
Set Transformer aggregation	Update the two embeddings using attention, then pool	SAB rows follow input order; PMA is invariant	Adds parameters and computation

Mean Pooling

Mean pooling is the simplest order-independent baseline. It represents the pair by the element-wise average

$$\mathbf{z}_{\text{mean}} = \frac{\mathbf{z}_A + \mathbf{z}_B}{2}. \tag{3.5}$$

Swapping the two inputs gives the same vector automatically, so this method is invariant by construction. The averaged vector is passed through this method's prediction head. Different pairs can share the same average; the possible information loss is discussed in the next subsection [37, 47].

Order-Averaged Concatenation

Concatenation keeps the two embeddings side by side, but one ordered concatenation depends on which molecule is listed first. The adapted baseline therefore evaluates both $[\mathbf{z}_A \,|\, \mathbf{z}_B]$ and $[\mathbf{z}_B \,|\, \mathbf{z}_A]$ through the same prediction head within that model. Each ordered input has 512 features. Let g_c denote the head, which returns one raw score, or *logit*, per descriptor. The final scores are calculated as

$$\hat{\mathbf{y}}_c = \text{sigmoid}\left(\frac{g_c([\mathbf{z}_A \,|\, \mathbf{z}_B]) + g_c([\mathbf{z}_B \,|\, \mathbf{z}_A])}{2}\right). \tag{3.6}$$

Swapping the inputs exchanges the two terms in the sum, leaving the prediction unchanged. Both embeddings remain available to each ordered evaluation, but the method requires two head evaluations. Averaging logits before sigmoid is different from averaging probabilities after sigmoid. This is an adapted baseline based on Sisson et al.'s pairwise design, rather than a reproduction of their complete model [33].

Set Transformers and Attention-Based Pooling

The Set Transformer receives the embeddings as a two-row matrix $X = [\mathbf{z}_A; \mathbf{z}_B] \in \mathbb{R}^{2\times 256}$. Attention allows each row to use information from both rows before pooling [22, 39]. A Self-Attention Block (SAB) uses a Multihead Attention Block (MAB), which contains attention, residual additions, a feedforward transformation applied to each row, and layer normalization:

$$\begin{aligned} H &= \text{LayerNorm}\big(Q + \text{Multihead}(Q, C, C)\big), \\ \text{MAB}(Q, C) &= \text{LayerNorm}\big(H + \text{rFF}(H)\big). \end{aligned} \tag{3.7}$$

Here, Q supplies the query rows and C supplies the rows used as keys and values. For self-attention, both are the same input: $\text{SAB}(X) = \text{MAB}(X, X)$. The residual additions keep the incoming

representation available alongside the learned update. The row-wise feedforward network transforms each row in the same way.

Swapping the input rows swaps the corresponding SAB output rows. This is *permutation equivariance*; it does not mean that the ordered output matrix stays unchanged. The implementation uses two SABs, each with four attention heads and a feedforward network of size $256 \to 512 \to 256$. It does not assign positional encodings to the rows.

Pooling by Multihead Attention (PMA) then combines the two updated rows:

$$\begin{aligned} U &= \mathrm{SAB}_2\big(\mathrm{SAB}_1(X)\big), \\ \mathbf{z}_{\mathrm{set}} &= \mathrm{MAB}\big(S, \mathrm{rFF}(U)\big). \end{aligned} \tag{3.8}$$

where S is one learned seed row. This row attends to the transformed inputs and returns a single 256-dimensional representation regardless of their order [22]. The implementation adapts these blocks using ReLU feedforward layers and normalization after each residual addition.

Mean pooling and Set Transformer aggregation each pass a 256-dimensional vector to their own prediction head, whereas concatenation passes 512 features per order. Each head has a 128-unit ReLU hidden layer and 92 output logits. Heads are trained separately across methods and runs; only the two orders within a concatenation model share a head. The models therefore differ in parameter capacity as well as aggregation.

The attention weights combine representations for descriptor prediction. They do not describe physical molecular interactions or receptor mechanisms. Whether the additional capacity improves prediction is an empirical question under the shared training and evaluation procedure.

3.1.5 Representation Trade-offs

The three aggregation methods differ in how they combine the embeddings and how much computation they require. Mean pooling is simple and order independent, but different pairs can produce the same average [47, 37]. It remains a useful baseline because the averaged representation may contain enough information for descriptor prediction.

Order-averaged concatenation keeps both embeddings available to the prediction head, but requires two ordered head evaluations to obtain an order-independent prediction. Set Transformer aggregation uses additional learned operations to update the embeddings before pooling [22]. This adds parameters and computation. The comparison therefore examines whether the additional com-

plexity provides a useful improvement under the same training and evaluation conditions.

Why Averaging Can Lose Information

Averaging two molecule embeddings can make structurally different pairs indistinguishable to the model. In one dimension, the pairs $(2, 8)$ and $(5, 5)$ both average to 5, so the averaged representation cannot distinguish those pairs. This example demonstrates a possible information loss; it does not show that such collisions are common or dominant in the present dataset. The actual usefulness of mean pooling remains an empirical question. The order-averaged concatenation baseline retains the two embeddings side by side while correcting for the two possible input orders, and the Set Transformer provides another order-invariant way to combine them by learning statistical relationships before pooling [22]. A full derivation of the symmetry argument is given in Appendix A.1.

3.1.6 Sparse Labels and Label Reliability

The blend targets form a sparse multi-label matrix. Each cell represents one blend and one descriptor, and most descriptors are unlisted. During training, the many negative cells can dominate the loss. During evaluation, a model can obtain a low Hamming loss by predicting mostly negatives while missing positive descriptors. Appropriate loss and evaluation methods are therefore needed.

The targets come from catalog recommendations rather than a controlled sensory panel. Reliability checks describe the consistency of these records, not their perceptual correctness. The single-molecule pretraining targets retain the available-source union of mapped ingredient labels. This is separate from the following evidence checks.

Blend recommendation evidence checks support for a specific blend descriptor in each component direction. Let E_A contain the descriptors under which A's GoodScents record recommends B, and E_B those under which B's record recommends A. Their intersection, $E_A \cap E_B$, contains descriptors supported in both directions. Repeated listings in one direction do not provide evidence in the other direction. This rule reduces reliance on a single recommendation, but can exclude recorded positives when reverse evidence is missing or unrecovered. Both records also belong to the same catalog, so their agreement is not independent sensory validation.

Cross-catalog agreement is a separate descriptor-level check. Cohen's kappa measures agreement between GoodScents and Leffingwell ingredient annotations while accounting for agreement expected by chance [40]. It is computed using molecules in the training partition that appear in

both catalogs. The study uses $\kappa \geq 0.21$ as its operational consistency threshold.

An unlisted blend descriptor is encoded as a negative target under a closed-world assumption. This supports supervised learning but does not establish sensory absence, because catalogs may omit descriptors.

A cell-level eligibility mask determines which targets contribute to training and evaluation. The target y_{bl} records whether descriptor l is listed for blend b. The mask m_{bl} records whether that cell is used: 1 means include and 0 means ignore.

Eligible Positive and Eligible Negative Cells

Eligibility determines whether a blend–descriptor cell contributes to training and evaluation. Under the primary condition, the study uses the following definitions:

- **Eligible positive.** A descriptor is recorded for the blend, has blend recommendation evidence from both component directions, and passes the descriptor-level consistency gate of $\kappa \geq 0.21$. The cell has target $y_{bl} = 1$ and mask $m_{bl} = 1$.
- **Eligible negative.** A descriptor is unlisted for the blend and passes the same descriptor-level consistency gate. The cell has target $y_{bl} = 0$ and mask $m_{bl} = 1$. Blend recommendation evidence is not required for an unlisted negative. Its negative status follows the closed-world assumption and does not establish sensory absence.

Both types contribute to the relevant loss and evaluation. Eligible negatives allow false-positive predictions to be counted. Cells that fail the applicable eligibility rule are masked with $m_{bl} = 0$. A recorded positive with insufficient evidence remains excluded rather than being converted into a negative. Masking excludes individual cells while retaining the blend rows and descriptor columns.

The expanded evidence condition also includes recorded positives with blend recommendation evidence from only one component direction, provided their descriptors pass the consistency gate. Recorded positives with no recovered blend recommendation evidence remain masked. The full-label condition uses all target cells without these evidence restrictions. Chapter 4 provides the complete rules.

3.2 Conceptual Framework

The conceptual framework organizes the study using the Input-Process-Output (IPO) model. It connects the molecular structures and catalog records to the computational steps used for descriptor prediction and evaluation. Figure 3.1 presents an overview of this framework.

The process includes data preparation, component-disjoint partitioning, molecular graph encoding, pair aggregation, and multi-label prediction. The two molecules pass through the same pretrained, frozen encoder. Their embeddings are then combined using mean pooling, order-averaged concatenation, or Set Transformer aggregation. Catalog labels and eligibility masks guide training and evaluation. The outputs include descriptor scores and predictions for each blend, which are evaluated under the study's declared conditions. Chapter 4 describes the detailed procedures and settings.

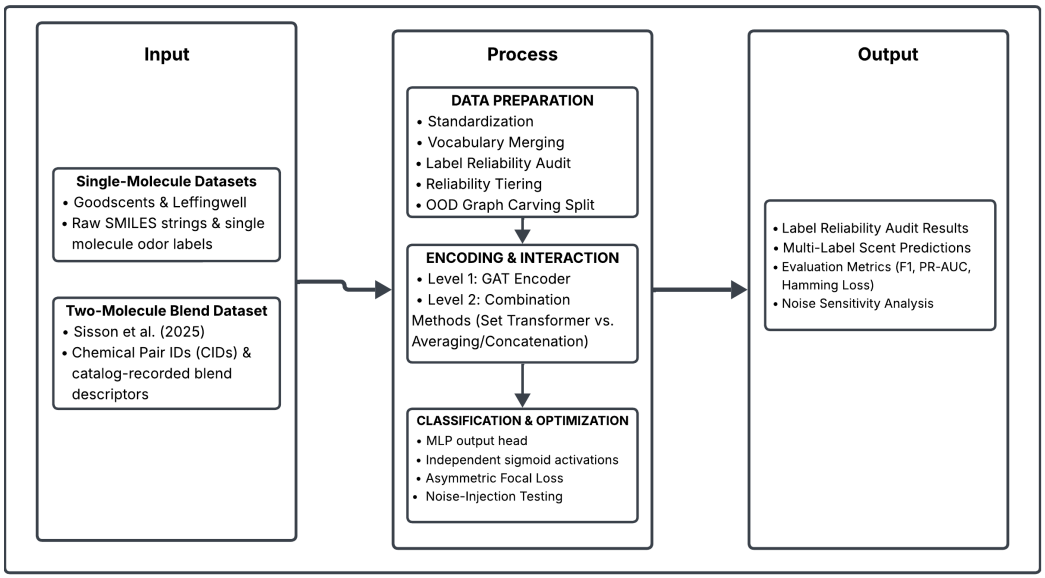


Figure 3.1: Input-Process-Output (IPO) Model.

3.2.1 Input Phase

The inputs include molecular structures and catalog-recorded descriptors from GoodScents and Leffingwell, accessed through Pyrfume [7]. The structures are represented as SMILES strings and

converted into molecular graphs. Available mapped ingredient labels are combined across the sources to form the single-molecule pretraining targets.

Blend-level records come from the local export obtained from the public repository associated with Sisson et al. [33]. These records identify the two components and their catalog-recorded blend descriptors. Archived recommendations from the component records provide the blend recommendation evidence used to check whether both directions support the same descriptor.

Molecular structures provide the model's prediction inputs. Ingredient and blend labels provide supervision, while blend recommendation evidence and catalog agreement determine eligibility for the reliability-aware conditions. The source organization and preparation procedures are described in Chapter 4.

3.2.2 Process Phase 1: Data Preparation and Graph Carving

The structures first pass through the study's standardization and filtering procedure. RDKit is used to parse the SMILES strings, remove salts, and produce canonical isomeric SMILES for matching structures across sources. Consistent molecular representations are needed before graph construction [42, 24].

The inclusion rules retain molecules containing C, H, O, N, S, F, Cl, Br, and I, with molecular weights between 20 and 600 g/mol. These are operational boundaries for the selected dataset. Passing the filters does not establish that a molecule is odor-active or safe, and the molecular-weight limits are not treated as universal limits of human perception.

Molecules are assigned to structure-based training, validation, and test partitions. Graph carving retains a blend only when both components belong to the same partition. This produces component-disjoint retained pairs, although structurally similar molecules can remain across partitions [12].

Descriptor names are harmonized, and the blend vocabulary is selected from the retained training data only. The separate catalog-consistency gate is also calculated using training-partition molecules shared by GoodScents and Leffingwell. These decisions do not use validation or test labels to select the vocabulary or estimate the gate.

Blend recommendation evidence from both component directions supports a specific blend descriptor. Together with the consistency gate, it determines which cells enter the primary eligibility mask. Eligible unlisted negatives follow the closed-world assumption, while listed positives with insufficient evidence remain masked. The single-molecule source-label union remains a separate

preparation step. Chapter 4 provides the complete rules and settings.

3.2.3 Process Phase 2: Encoding and Pair Representation

Each molecular graph is converted into a 256-dimensional embedding by the shared GATv2 encoder described in Section 3.1.3. Graph-attention methods combine information from atoms and their neighborhoods, while attention readout produces a molecular representation [45].

The encoder is pretrained on training-partition single molecules and then frozen for blend training. Both components and all three aggregation methods therefore use the same encoder parameters.

Mean pooling averages the two embeddings. Order-averaged concatenation passes both input orders through its prediction head and averages the resulting logits before sigmoid. This is an adaptation of the pairwise design of Sisson et al. [33]. Set Transformer aggregation updates the two embeddings through attention before pooling them into one representation [22].

These methods learn or construct statistical representations for descriptor prediction. Their computations are not interpreted as physical molecular interactions or receptor mechanisms.

3.2.4 Process Phase 3: Classification and Optimization

Each aggregation method has a separately trained prediction head. Mean pooling and Set Transformer aggregation use heads with dimensions $256 \rightarrow 128 \rightarrow 92$. Order-averaged concatenation uses $512 \rightarrow 128 \rightarrow 92$ and applies the same within-model head to both ordered concatenations.

The head produces one logit per descriptor. Sigmoid converts each logit into a score between zero and one, allowing several descriptors to be predicted for the same blend. This follows the multi-label nature of odor-description tasks [29]. These are separate descriptor scores, not a joint probability distribution or evidence of calibration.

Blend training uses the masked asymmetric focal loss defined in Equation 3.11. Asymmetric focal objectives have been investigated in graph-based prediction [18]; the particular parameters and masking rules used here are specified in Chapter 4.

Because the encoder is frozen, prediction errors update only the trainable aggregation and prediction modules. The controlled corruption experiment changes eligible training targets in the primary condition, while validation and test targets remain unchanged. This examines sensitivity to the declared label-noise procedure [34].

3.2.5 Output Phase

The models produce one score for each of the $L = 92$ blend descriptors. A decision cutoff selected using validation data converts these scores into binary predictions. The selected cutoff is held fixed during test evaluation.

Performance is evaluated on the cells included by each condition's mask. Macro F1 is the primary metric, supported by micro F1, macro and micro average precision, and Hamming loss. Multiple metrics are needed because label imbalance can affect their interpretation [27, 41].

Macro scores include only descriptors meeting the declared eligible positive and negative support rule in the evaluated partition. The model still retains all 92 output scores. Hamming loss is interpreted alongside an all-negative reference.

The framework also supports summaries of blend recommendation evidence, cross-catalog agreement, and sensitivity to training-label corruption. Chapter 4 defines the evaluation and statistical comparison procedures. These outputs support an ongoing comparison rather than establish sensory validity or a universally superior aggregation method.

3.3 Technical Background

The implementation uses RDKit to check and prepare molecular representations, PyTorch Geometric to process sparse graphs, and masked losses to train on the selected target cells. These checks establish compatibility with the processing rules, not experimental proof of a molecule's existence, odor activity, or safety. This section explains these tools and calculations, followed by the metrics used to evaluate descriptor predictions.

3.3.1 Cheminformatics Standard (RDKit): SMILES Parsing and Sanitization

Molecular structures are stored as SMILES strings and converted into graphs using RDKit. SMILES provides a compact representation of atoms, bonds, and other structural information [42, 24].

During parsing and sanitization, RDKit computes structural properties and checks whether the representation satisfies its supported rules. Relevant operations include valence checking, ring identification, kekulization, aromaticity assignment, and the calculation of conjugation and hybridization

properties. These are examples of sanitization operations rather than an exhaustive four-step sequence.

The resulting atom and bond properties are used to construct the graph features. Structures that cannot be processed under the study's declared rules are excluded. Additional element and molecular-weight filters define the dataset's inclusion boundaries.

These checks establish compatibility with the chosen representation and processing rules. They do not experimentally establish that a structure exists, is odor-active, is safe, or will produce a particular sensory response. The exact filtering procedure is documented in Chapter 4.

3.3.2 Deep Learning Stack (PyTorch / PyTorch Geometric): Sparse Graph Convolutions

After sanitization, each molecule is formatted as an undirected graph $G = (V, E)$, where atoms are nodes (V) and covalent bonds are edges (E). A dense $N \times N$ adjacency matrix could represent this connectivity, but for molecules, where most atoms bond to only a few neighbors, this wastes memory, scaling at $\mathcal{O}(N^2)$ for structures that are inherently sparse. The architecture instead runs on PyTorch Geometric, which handles molecular graphs as sparse attributed objects rather than dense arrays [11].

Topology is stored in Coordinate (COO) format: a sparse two-row index matrix C of size $2 \times 2E$, where the first row lists source node indices and the second lists receiver node indices [4]. Since chemical bonds are physical and undirected, every source node is also a receiver, so $2E$ entries are needed rather than E — each bond is represented in both directions so that message-passing can update both of its atoms. Node states are updated by the GATv2 computation described in Section 3.1.3, using the sparse connectivity and associated bond features at each layer.

3.3.3 Multi-Label Mathematical Formulation: Independent Sigmoids and Sparsity Mitigation

A blend can have several recorded descriptors at the same time. The task is therefore multi-label rather than a choice among mutually exclusive classes [29]. For example i and descriptor k, the prediction head produces a logit s_{ik}. Sigmoid converts it into a descriptor score:

$$p_{ik} = \sigma(s_{ik}) = \frac{1}{1 + \exp(-s_{ik})}. \tag{3.9}$$

The target y_{ik} records whether the descriptor is listed. The mask M_{ik} determines whether the cell contributes to learning: $M_{ik} = 1$ includes it and $M_{ik} = 0$ excludes it. During encoder pretraining, the mask excludes unknown targets. During blend training, it follows the evidence condition defined in Chapter 4.

Sparse targets contain relatively few positive cells. The many negative cells can otherwise dominate learning, so loss design and evaluation must account for imbalance [41].

Encoder pretraining uses masked binary cross-entropy. For a batch of N examples and L descriptors, this is:

$$\mathcal{L}_{\mathrm{BCE}} = -\frac{\sum_{i=1}^{N}\sum_{k=1}^{L} M_{ik}\left[y_{ik}\log p_{ik} + (1-y_{ik})\log(1-p_{ik})\right]}{\sum_{i=1}^{N}\sum_{k=1}^{L} M_{ik}}. \tag{3.10}$$

Blend training uses asymmetric focal loss. Such objectives apply different focusing to positive and negative terms and have been studied in graph-based prediction [18]. The form used in this implementation is defined below; its parameter choices are this study's settings.

First, define the margin-adjusted negative score:

$$q_{ik} = \max(p_{ik} - \delta, 0).$$

The per-cell loss and its masked average are:

$$\begin{aligned} \ell_{ik} &= -\Big[y_{ik}(1-p_{ik})^{\gamma_p}\log p_{ik} \\ &\qquad + (1-y_{ik})q_{ik}^{\gamma_n}\log(1-q_{ik})\Big], \\ \mathcal{L}_{\mathrm{ASL}} &= \frac{\sum_{i=1}^{N}\sum_{k=1}^{L} M_{ik}\ell_{ik}}{\sum_{i=1}^{N}\sum_{k=1}^{L} M_{ik}}. \end{aligned} \tag{3.11}$$

The implementation uses $\gamma_p = 0$, $\gamma_n = 2$, and $\delta = 0.05$, without class-frequency alpha weights. The positive exponent leaves the positive term unfocused, while the negative exponent and margin reduce the contribution of easy negative cells. Numerical safeguards are used when calculating logarithms.

The denominator counts eligible cells rather than all possible cells. A batch must contain at least one eligible cell for this loss to be calculated. The loss addresses imbalance, while the mask determines which available targets are used. Neither establishes that catalog annotations are perceptually correct.

3.3.4 Evaluation Metrics for Sparse Multi-Label Outputs

Macro F1 is the primary evaluation metric. It calculates F1 separately for each supported descriptor and averages the values with equal weight. Micro F1 instead pools true positives, false positives, and false negatives before calculating one score [27].

In this study, a descriptor enters macro reporting only when the evaluated partition contains at least one eligible positive and one eligible negative for it. This reporting rule does not remove output columns from the model. Chapter 4 explains the support requirement and its documented amendment.

Average precision is reported as a supporting ranking metric. The implementation uses `average_precision_score`, which weights precision by changes in recall rather than applying trapezoidal integration to the precision–recall curve. Macro average precision averages the supported descriptor scores; micro average precision pools all eligible cells. Precision–recall evaluation is useful alongside F1 in imbalanced odor-prediction tasks [41].

Binary predictions are obtained using a global cutoff selected for each method and condition from validation performance averaged across the five seeds. The cutoff is then held fixed for test evaluation. Checkpoint selection is a separate decision based on validation loss. Chapter 4 provides the selection procedures.

Hamming loss is the fraction of eligible cells predicted incorrectly. Because most targets are negative, it must be interpreted alongside an all-negative reference. For evaluation mask M_{ik}, that reference is:

$$\mathrm{HL}_{\mathrm{neg}} = \frac{\sum_{i=1}^{N}\sum_{k=1}^{L} M_{ik} y_{ik}}{\sum_{i=1}^{N}\sum_{k=1}^{L} M_{ik}}. \tag{3.12}$$

This equals the positive prevalence among eligible cells: predicting every cell as negative makes an error on each eligible positive. It is calculated separately for each partition and evidence condition.

Uncertainty in model differences is examined using paired bootstrap resampling of whole blend rows, retaining their descriptor targets and masks. The same sampled rows are used for the compared methods and seeds. Shared molecular components within the test partition can create dependence between rows, which limits the interpretation of this procedure. Chapter 4 gives the detailed statistical protocol.

Chapter 4

Methodology

This chapter describes how the catalog-derived data were prepared, how the label reliability masks were built, how component-disjoint partitions were formed, and how three aggregation methods were trained and evaluated under matched conditions. It also defines the robustness conditions, metrics, and paired comparisons used to answer the research questions.

4.1 Research Design

This study uses a quantitative experimental design on a multi-label molecular graph classification task. It compares three ways of combining the learned embeddings of the two molecules in a blend: mean pooling, order-averaged concatenation, and Set Transformer aggregation. Experiments A and B provide the matched aggregation comparison for RQ1, the blend-type analysis addresses RQ2, and controlled label corruption addresses RQ3.

Table 4.1 states what is varied and what is held fixed. The aggregation method is the primary varied factor. The label condition and the corruption level are robustness factors, and the catalog-label pattern of a blend is used only for a subgroup analysis.

Figure 4.1 gives an overview of the implemented pipeline.

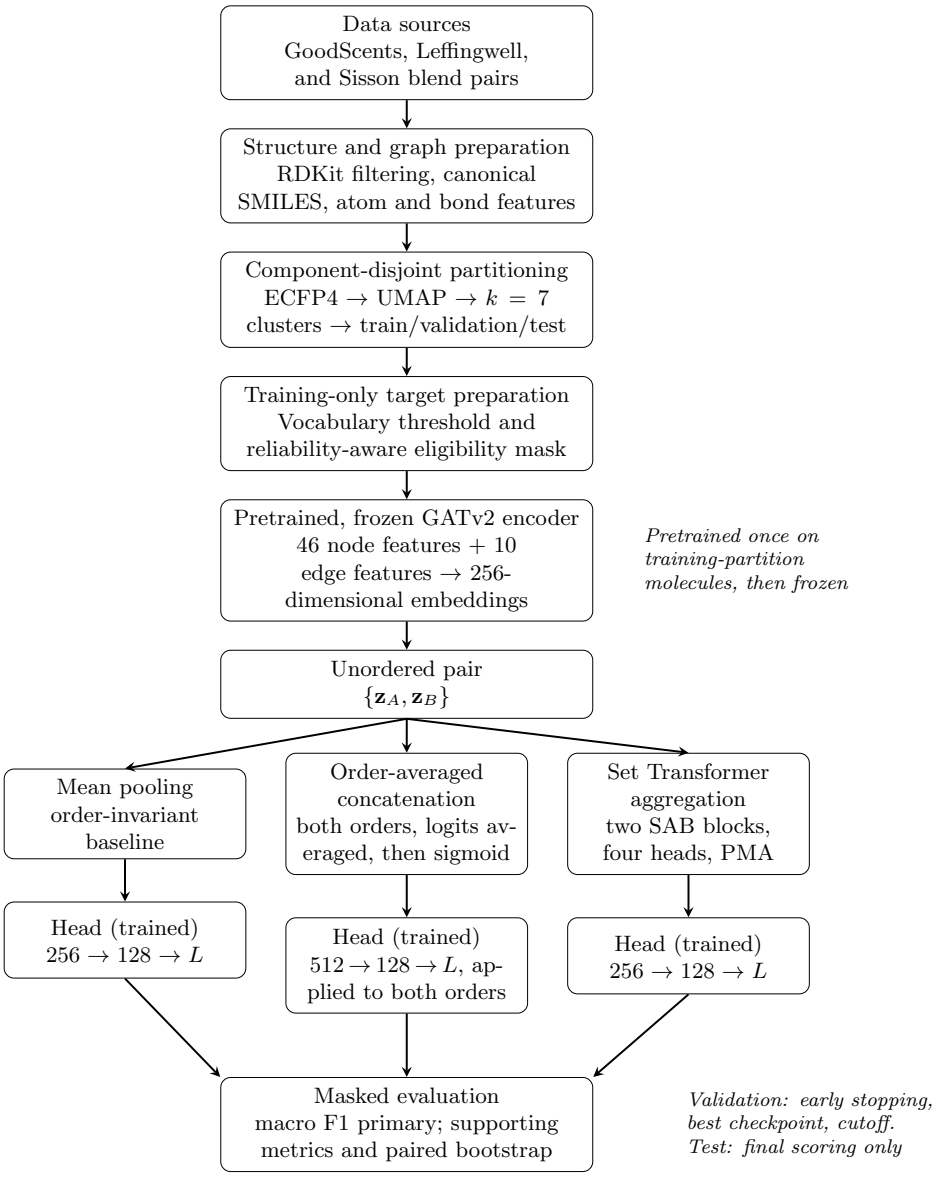


Figure 4.1: Overview of the implemented pipeline. The three aggregation methods receive the same frozen embeddings and the same evaluation protocol, and each has its own separately trained prediction head.

Table 4.1: Varied factors and matched controls.

Category	**Treatment**
Aggregation method	Primary varied factor (three levels)
Label condition and corruption level	Robustness factors
Catalog-label pattern	Subgroup analysis
Molecular encoder	One pretrained, frozen artifact shared by all methods
Prediction head	Trained separately for each method; input size differs (concatenation uses 512 inputs)
Partitions and inputs	Same across methods
Loss, optimizer, stopping rule, training budget	Same across methods
Random seeds and corruption draws	Same seed IDs across methods; variation across seeds is reported separately (Section 4.8)

4.2 Data Acquisition and Preparation

This section describes the data sources, how they were joined, and how the model inputs (molecular structures) and targets (scent labels) were prepared. Molecular structures were converted into graphs, and scent labels were organized into a fixed set of labels.

4.2.1 Data Sources and Provenance

The study used four kinds of records (Table 4.2). The single-molecule sources are GoodScents and Leffingwell, both accessed through Pyrfume. They are independently maintained reference catalogs for the perfumery and flavor industries, and each record lists a molecule, its structure, and the scent labels assigned to it. The Leffingwell labels are described as assigned by olfactory experts, usually practicing perfumers [21]. The blend data came from the public repository of Sisson et al. [33], and archived GoodScents pages supplied the direction of each blending recommendation. The single molecule datasets were first combined. Their structures were then standardized and filtered following the specified criteria. After filtering, records that resolved to the same SMILES were consolidated into one master molecule table, while retaining the original GoodScents and Leffingwell stimulus IDs and CIDs, as well as the scent labels recorded by each catalog.

Structure filtering. Before molecules were matched across sources, structures were sanitized with RDKit: salts were removed, molecules containing elements other than C, H, O, N, S, F, Cl, Br, and I were excluded, and molecules outside 20–600 g/mol were excluded. These rules are inclusion heuristics for typical aroma chemicals. This step used structure only; no label was consulted.

Table 4.2: Data sources.

Source	**Data organization**	**Purpose in this study**
GoodScents (single molecules)	Scent labels per stimulus; a stimulus-to-CID metadata table; a CID-to-structure table	Reference labels and pretraining; source-consistency check
Leffingwell (single molecules)	One indicator column per scent label for each stimulus, joined through stimulus and CID	Reference labels and pretraining; source-consistency check
Sisson export (blend pairs)	Two component columns (CID.1 and CID.2) with a list of scent labels for each pair; component identities were matched and duplicate pairs removed	Blend targets
Archived GoodScents pages	Each component page lists suggested blending partners under scent label headings; recommendations were collected while recording which component made each suggestion	Evidence for two-page scent label intersection (Section 4.3)

Join key. The two single-molecule catalogs were merged into one master dictionary using the isomeric SMILES string as the key. Every SMILES string in both catalogs and in the blend dataset was canonicalized with RDKit so that the same molecule written in two ways matched correctly. SMILES was chosen over the PubChem CID because some CIDs in the source files are negative or non-standard, and SMILES must be canonicalized anyway when molecules are converted to graphs. Blend components in the Sisson et al. dataset, which are identified by CID, were linked to the master dictionary by canonicalizing their SMILES in the same way. Duplicate records, components that could not be joined or were filtered out, self-pairs, and scent labels with no match in the vocabulary were handled at this step; their counts are reported in Chapter 5.

Blend records. The Sisson et al. blend data came from blender recommendations crawled from GoodScents molecule pages [33]. Each page lists recommended partners under scent label headings, and because the two molecules' pages were crawled separately, the same pair and scent label can appear from one direction or from both. This study kept the direction of each listing so that the two-page scent label intersection (Section 4.3) could be computed.

Multiple crawled pages refer to the same chemical molecule. These molecules are treated as one molecule identity when their structures resolve to the same canonical isomeric SMILES. The original page records are retained for traceability, and recommendations from those pages are grouped under the molecule direction. Multiple pages for one component do not count as support from both components. The same scent label must be supported from each component's side to meet the

two-page intersection rule.

4.2.2 Catalog Targets and Their Limits

Catalog-recorded scent labels serve as proxy targets for supervised learning. They are curated reference records with some noise and subjectivity [6]. This study treats them as a practical proxy for perceived scent, estimates how consistent they are, and examines how model comparisons change as label reliability changes.

4.2.3 Target Vocabulary

Two vocabularies are kept apart. The *standardized reference vocabulary* has 104 scent labels (Appendix A.2.3), obtained after spelling variants were merged, and is used for pretraining.

Standardization. The raw GoodScents and Leffingwell labels contain spelling variants of the same scent label (for example, anisic and anise, or caramel and caramellic), so this study adopted the open-source canonicalization dictionary of Sisson et al. [33]. scent labels the dictionary did not cover were inventoried and reviewed.

Final target vocabulary. After graph carving, a standardized descriptor was retained if it was positive in at least 30 full-label blends in training partition. This training-only rule retained 92 descriptors ($L = 92$) from the 104-descriptor reference vocabulary. Validation and test labels were not used to select the vocabulary.

Molecules without reference labels. Training molecules with no mapped reference labels kept their structures, and their targets were marked unknown (masked). The model has no "unknown" or "odorless" class.

4.3 Label Reliability Mask

Not every recorded scent label is equally trustworthy, and not every missing label is a true absence. For each blend–scent label cell (one blend and one scent label in the label matrix), the study decided whether that cell is used in the stricter eligibility-mask condition.

4.3.1 Consistency Evidence

Two kinds of evidence are used to measure consistency.

- **Two-page scent label intersection (cell level).** For a pair of molecules A and B, let E_A be the scent labels under which A's page recommends B, and let E_B be the scent labels under which B's page recommends A. A scent label has primary support for the pair when it is in both, $E_A \cap E_B$. For example (illustrative), if $E_A = \{\text{sweet, fruity}\}$ and $E_B = \{\text{fruity, woody}\}$, only "fruity" is supported. Requiring both pages reduces reliance on one-sided recommendations, at the cost of excluding valid labels when the reverse recommendation is missing or a page is incomplete, and both pages come from the same catalog.

- **Scent label-level consistency (scent label level).** Agreement between GoodScents and Leffingwell was measured for each scent label using Cohen's kappa, computed only on training-partition molecules that occur in both catalogs. A scent label with $\kappa \geq 0.21$ is called consistent.

4.3.2 Eligibility Mask

Table 4.3 defines the eligibility mask $M_{ij} \in \{0, 1\}$ for blend i and scent label j.

Table 4.3: Cell-level rules for the eligibility mask.

Target	**scent label** κ	**Page support**	**Interpretation**	**Mask**
Listed	≥ 0.21	2 (both pages)	Eligible positive	M = 1
Listed	≥ 0.21	1 (one page)	Not confirmed	M = 0
Listed	≥ 0.21	0 (not recovered)	Support unknown	M = 0
Listed	< 0.21	Any	Agreement too weak	M = 0
Unlisted	≥ 0.21	Not applicable	Eligible negative (closed-world assumption)	M = 1
Unlisted	< 0.21	Not applicable	Negative status not reliable	M = 0

Under the primary eligibility-mask condition, entries marked 1 are included in the masked training loss and primary evaluation metrics. Entries marked 0 are excluded from both and are not automatically treated as negative labels. Because eligible unlisted scent labels enter as negatives, the masked test set contains both positives and negatives, so a model that predicts an unrecorded scent label is penalized as a false positive.

Closed-world assumption. Unlisted scent labels are treated as absent so that they can serve as negative labels for training and evaluation. A catalog may simply have left a scent label out, which the eligibility mask reduces but does not remove.

4.3.3 Experimental Label Conditions

Three label conditions are used:

- **Primary (eligibility-mask) condition.** Only cells with $M_{ij} = 1$ under Table 4.3 are used.
- **Expanded either-page sensitivity.** Same as the primary condition, except that listed positives with a page support of 1 are also counted as positives, to test how much the results depend on the stricter primary requirement.
- **Full-label diagnostic.** Every cell is used ($M_{ij} = 1$ for all i, j), with listed scent labels as positives and unlisted scent labels as negatives.

Masked loss. Masking excludes cells from the loss and scoring; in the primary condition a blend is kept as long as it has at least one eligible cell. The loss is computed only over eligible cells:

$$L_{\text{masked}} = \frac{\sum_i \sum_j M_{ij}\, \ell_{ij}}{\sum_i \sum_j M_{ij}} \tag{4.1}$$

where ℓ_{ij} is the per-cell loss (Section 4.4.3). With $M_{ij} = 1$ everywhere this is the ordinary mean loss. The same mask is applied when metrics are computed (Section 4.7).

Limited support. A scent label is included in macro-averaged scores only if the evaluated partition has at least one eligible positive and at least one eligible negative cell for it. This working rule was adopted before the main model results were evaluated, in place of the five-per-side rule of the approved manuscript. Excluded scent labels are listed in the report, and the output layer keeps all L outputs.

4.4 Model Architecture and Training

All three models use the same frozen molecular graph encoder, and each has its own separately trained prediction head. They differ only in the step that combines the two molecule embeddings.

Each molecule is turned into a graph and encoded into a vector, the two vectors are combined by the aggregation method, and the combined vector is converted into one probability per scent label.

4.4.1 Shared Graph Encoder

Each molecule is a graph in which atoms are nodes and bonds are edges. A Graph Attention Network (GAT) encoder turns each graph into a 256-dimensional vector $\mathbf{z}$, so a blend yields two embeddings $\mathbf{z}_A$ and $\mathbf{z}_B$, and the same encoder weights are applied to both molecules.

The encoder receives the 46-dimensional atom features and 10-dimensional bond features listed in Appendix A and uses three edge-aware GATv2 layers. The first two layers use four attention heads with 128 features per head, and the final layer uses one head to produce a 256-dimensional node representation. A residual connection is used in the second layer, followed by an attention readout that gives one vector per molecule. The attention computation is defined in Section 3.1.3.

Pretraining. The encoder was first pretrained on single-molecule data from the training partition only (Section 4.5), with a portion set aside for internal validation. A temporary head with 104 sigmoid outputs (one per standardized reference scent label) was trained with masked binary cross-entropy, so molecules with unknown targets were skipped and did not contribute to the loss. The best checkpoint was restored, the temporary head was removed, and the encoder weights were saved and frozen.

Frozen and trainable parts. The encoder is frozen during the blend-level comparison, so all three variants receive embeddings from the same encoder weights and only the aggregation module and prediction head are trained. This keeps the comparison focused on how the two embeddings are combined.

4.4.2 Aggregation Variants

Table 4.4 summarizes the three variants.

Mean pooling. The two embeddings are averaged to produce one blend embedding:

$$\mathbf{z}_{\text{blend}} = \tfrac{1}{2}(\mathbf{z}_A + \mathbf{z}_B) \in \mathbb{R}^d. \tag{4.2}$$

Order-averaged concatenation. The two embeddings are placed side by side, $[\mathbf{z}_A \,|\, \mathbf{z}_B] \in \mathbb{R}^{2d}$, and the model is run on both (A, B) and (B, A). The two logits are averaged before one sigmoid

Table 4.4: The three aggregation variants.

Variant	**Operation**	**Order handling**	**Head input**
Mean pooling	Average the two molecule embeddings	Invariant by construction	256
Order-averaged concatenation	Run both ordered concatenations through the head, average the logits, then apply one sigmoid	Explicit symmetrization	512 (two passes)
Set Transformer aggregation	Two self-attention blocks (SAB), then pooling by multihead attention (PMA)	SAB is equivariant; PMA gives an invariant output	256

is applied. This adapts the pairwise design of Sisson et al. [33] and uses this study's encoder, loss, and partitions.

Set Transformer aggregation. The pair $\{\mathbf{z}_A, \mathbf{z}_B\}$ is treated as a set and passed through two self-attention blocks (SABs), each with four attention heads and a 512-unit feedforward layer. A pooling by multihead attention (PMA) layer with one learned seed vector then combines the two outputs into a single blend embedding. The implementation is based on the Set Transformer modules of Lee et al. [22].

The number of trainable parameters and the training and inference time of each model are reported with the results.

4.4.3 Output Head and Loss

Each prediction head is a multilayer perceptron mapping $256 \to 128 \to 92$, where $L = 92$ is the number of target scent labels (Section 4.2.3). For order-averaged concatenation the first layer takes 512 inputs. The output layer applies an independent sigmoid to each of the L outputs, so scent labels are predicted independently. Blend-level training uses the asymmetric focal loss of Equation 3.11 with $\gamma_{\text{pos}} = 0$, $\gamma_{\text{neg}} = 2$, a negative margin of 0.05, no class-frequency weights, and a mean over eligible cells; its per-cell values enter the masked loss of Section 4.3.3. Binary cross-entropy is used only for encoder pretraining.

Sparse blend annotations and uncertainty in some pair-scent entries are addressed with an eligibility mask, which excludes unsupported entries from the loss rather than treating them as absent. The asymmetric focal loss uses a positive focusing exponent of 0 and a negative focusing exponent of 2, reducing the relative influence of easy negative examples. The other numerical settings, including

Table 4.5: Blend-level training settings for the three variants.

Setting	**Value**
Optimizer	Adam
Learning rate	0.001
Weight decay	0
Learning-rate schedule	Constant
Batch size	128
Gradient clipping	1
Early stopping	Patience of 20 epochs; minimum improvement of 0.0001
Maximum epochs	200
Loss	Asymmetric focal ($\gamma_{\text{pos}} = 0$, $\gamma_{\text{neg}} = 2$, margin 0.05)

the optimizer, learning rate, batch size, and stopping controls, were fixed implementation choices rather than values optimized for this dataset. These settings were held constant across the three aggregation methods, with validation loss used for checkpoint selection and early stopping.

Seeds and checkpoints. Each variant was trained with the same five seeds (0 to 4), which change the prediction head's initialization and the order of the training data. For each run, the checkpoint with the lowest validation loss was saved and used for testing. Early stopping decides when training ends, and the lowest validation loss decides which epoch's weights are kept.

4.5 Component-Disjoint Partitioning and Graph Carving

This section explains how the data were divided into training, validation, and test sets so that no molecule in a training blend also appears in a validation or test blend.

4.5.1 Scaffold Check

Scaffold splitting assigns molecules that share a structural framework to the same partition and is one approach to evaluating molecular property prediction models (Fooladi et al., 2025). To assess it as a possible partitioning method, Bemis–Murcko scaffolds were computed with RDKit for the 5,712 canonical molecules in the master dataset. The molecule-to-scaffold ratio was calculated as the number of molecules divided by the number of distinct scaffolds.The molecule-to-scaffold ratio is

$$R = \frac{N_{\text{molecules}}}{N_{\text{scaffolds}}}. \tag{4.3}$$

The diagnostic identified 549 distinct scaffolds, giving a ratio of 5,712 / 549, or approximately 10.40. This ratio is an average and does not show how unevenly molecules are distributed across scaffolds. It is also above the study's stated diagnostic criterion of R ¡ 2.0, so the ratio alone does not show that scaffold splitting was unsuitable.

However, 2,609 of the 5,712 molecules (45.7%) have molecular graphs with no cycles; these are ring-free molecules. RDKit assigns all of them the same empty Bemis Murcko scaffold, even though their full molecular graphs differ. A strict scaffold split would therefore place these molecules in one group, which would contain nearly half of the master dataset and could make the desired partition sizes difficult to balance. Scaffold identity would also fail to distinguish the acyclic molecules from one another.

For this dataset, the study used ECFP4 fingerprints with Jaccard-based UMAP and K-Means clustering instead. This approach groups molecules using structural features beyond the empty scaffold. The choice is specific to this dataset and does not mean that scaffold splitting is generally unsuitable.

4.5.2 Cluster-Based Partitions

Molecules were grouped by structural similarity, and whole groups were assigned to one partition. Each unique molecule was represented by a 2048-bit binary ECFP4 fingerprint with radius 2. UMAP projected the fingerprints to two dimensions using Jaccard distance, 15 neighbors, a minimum distance of 0.1, and random seed 42. K-Means then formed $k = 7$ clusters using 20 initializations and random seed 42, following Guo et al. [15]. UMAP's default approximate neighbor search left two very small structures disconnected, so exact Jaccard nearest neighbors were computed and supplied to UMAP, and disconnected components were retained (infinite disconnection distance).

Clusters were assigned to training, validation, and test partitions using molecule counts only. All assignments giving at least one cluster to each partition were enumerated, and the one with the smallest total absolute deviation from a 70:15:15 molecule ratio was selected, with ties broken lexicographically by cluster identifier. The rule used no scent label identities, label frequencies, pair counts, or test labels. Once fixed, the partitions were not changed. The cluster-to-partition assignment, molecule counts, retained pairs, and scent label support are reported in Chapter 5.

4.5.3 The Graph-Carving Protocol

Each molecule was assigned to a partition based on its cluster. A blend was retained only if both of its molecules belonged to the same partition, and blends connecting molecules from different partitions were discarded. As a result, no molecule appears in more than one partition. Structurally similar molecules can still fall in different partitions if they belong to different clusters, and a molecule can appear in multiple blends within its own partition.

The validation set was used for early stopping, hyperparameter decisions, and selection of the decision threshold. The test set was kept separate and used only for the final evaluation.

4.5.4 Training-Only Vocabulary Filter

Carving changes how often each scent label occurs, so the final target vocabulary was chosen after partitioning, from training data only. A standardized scent label d was kept if it had at least $x = 30$ positive training blends:

$$d \in D_{\text{target}} \iff c_{\text{train}}(d) \geq x \tag{4.4}$$

where $c_{\text{train}}(d)$ is the number of training blends in which d is recorded as positive. The vocabulary was determined once from the full-label training partition and is shared across all label conditions, so changing the eligibility mask does not change the prediction targets. This gives the final vocabulary size L. The numbers of positive and negative labels per scent label in the validation and test sets are reported separately and were not used to select the vocabulary. scent labels with insufficient eligible test support are handled by the rule in Section 4.3.3.

4.6 Experiments

The three aggregation methods use the same partitions, encoder, loss function, optimizer, stopping rule, training budget, and evaluation metrics, and each is trained with the same five seeds (0, 1, 2, 3, and 4).

The main experiments consist of 3 aggregation methods $\times$ 5 seeds $\times$ 3 label conditions, giving 45 model runs. The controlled corruption experiment adds another 45 runs and reuses the 15 clean primary-condition runs as its reference.

4.6.1 Experiments A and B: Main Comparison (RQ1)

Experiment A trains each model in the full-label condition, and Experiment B trains each model in the primary eligibility-mask condition (Section 4.3.3). A third run uses the expanded either-page condition as a sensitivity check. All are evaluated on the held-out test partition and scored on the cells defined by their own mask, which shows how much the comparison depends on the label condition.

4.6.2 Experiment C: Controlled Label Corruption (RQ3)

To examine how the comparison changes as training labels become less reliable, labels were corrupted on purpose in the training partition only; validation and test labels were never changed. Corruption was applied to the eligible cells of the primary condition. For a noise level $p \in \{0.10, 0.20, 0.30\}$ and N eligible positive cells, $\lfloor pN \rfloor$ positives were flipped to absent and the same number of eligible negatives were flipped to present, which keeps the number of positives unchanged. Flips were drawn across the whole training matrix, not per scent label. For each noise level and seed, one corruption pattern was generated from a random stream seeded by the level and the seed, and shared across all three aggregation methods. The $p = 0$ reference is the uncorrupted Experiment B run. Models were retrained at each level and scored on the unchanged validation and test labels. The change in macro F1 from $p = 0$ to $p = 0.30$ is compared across methods with the paired bootstrap of Section 4.8.

4.6.3 Blend-Type Analysis (RQ2)

Each test blend is compared with the recorded scent labels of its two parent molecules, taken as the union of the two components' standardized GoodScents and Leffingwell scent labels, and assigned to one of three groups:

- **Catalog-label loss:** a scent label recorded for either parent is missing from the blend.
- **Catalog-label gain:** the blend has a recorded scent label that neither parent has.
- **Additive:** the blend shows neither loss nor gain.

The number of blends in each group is reported, and model performance is compared separately within each group. This is an exploratory subgroup analysis. If a group contains fewer than 5%

or more than 95% of the test blends, blends are instead grouped by the Jaccard overlap $J = |P \cap B|/|P \cup B|$ between the parents' scent label union P and the blend's scent labels B. Three fixed equal-width bins are used: low ($0 \leq J < 1/3$), middle ($1/3 \leq J < 2/3$), and high ($2/3 \leq J \leq 1$), with boundary values assigned to the upper bin. The bins were fixed before subgroup scoring and are not adjusted when a bin is sparse. Macro metrics are reported as unavailable for a bin without eligible positives.

4.7 Evaluation Metrics

4.7.1 Multi-Label Metrics

Let TP be a scent label correctly predicted present, FP a scent label incorrectly predicted present, and FN a present scent label that was missed.

- **Macro F1 (primary metric).** F_1 is computed for each scent label separately and then averaged with equal weight, so rare scent labels count as much as common ones [27].
- **Micro F1.** TP, FP, and FN are pooled across all scent labels before F_1 is computed; it favors common scent labels.
- **Average precision (PR-AUC).** The step-wise average precision (as in scikit-learn's average_precision_score, not trapezoidal integration of the precision–recall curve), which summarizes performance across all probability thresholds. Macro AP averages the per-scent label values over the scored scent labels, and micro AP is computed over all eligible cells pooled.
- **Hamming loss.** The fraction of evaluated cells that are predicted incorrectly.

Decision cutoff. For each method and label condition, one global cutoff converts predicted probabilities into present or absent labels. It is selected from the grid $\{0.05, 0.10, \ldots, 0.95\}$ as the value giving the highest validation macro F1 averaged across the five seeds, and is then applied unchanged to all five seeds at test.

4.7.2 Masked Metrics

All metrics are computed only over cells with $M_{ij} = 1$, using the mask of the condition being evaluated (all ones in the full-label condition). Macro metrics use only scent labels with enough

eligible support in the evaluated partition (Section 4.3.3). With a mask, Hamming loss is

$$\mathrm{HL}_M = \frac{\sum_{i=1}^{N} \sum_{j=1}^{L} M_{ij}\, \mathbf{1}(y_{ij} \neq \hat{y}_{ij})}{\sum_{i=1}^{N} \sum_{j=1}^{L} M_{ij}} \tag{4.5}$$

where N is the number of evaluated blends, L is the number of target scent labels, y_{ij} is the label, and $\hat{y}_{ij}$ is the prediction after applying the decision cutoff. Smaller values are better.

All-negative reference. Hamming loss rewards the many easy negatives, so each model's value is reported next to that of a baseline predicting every evaluated cell as absent. This baseline equals the fraction of evaluated cells that are positive:

$$D_M = \frac{\sum_i \sum_j M_{ij}\, y_{ij}}{\sum_i \sum_j M_{ij}} \tag{4.6}$$

D_M is computed separately for each partition and mask. It differs from mask coverage, the fraction of cells with $M_{ij} = 1$.

4.8 Statistical Comparison

Differences between models can reflect chance in which blends happen to be in the test set, or chance in training. The two sources are handled separately.

Variability across seeds. Each model was trained with the same five seeds (0 to 4). Seed-to-seed variation is summarized separately as the mean and spread across runs. Within each bootstrap replicate, every seed is evaluated separately on the resampled test blends, and the paired differences between two methods across the five matching seeds are then averaged.

Bootstrap over blends. scent label outcomes within one blend are related, so individual cells are not sampled independently. Whole test blends are resampled as rows, each keeping all its scent label labels and eligibility mask. From the N test blends, N rows were drawn with replacement 10,000 times, using random seed 42. All models and seeds are scored on the same resampled blends, giving a paired comparison. For replicate b, the difference between two methods is

$$\Delta(b) = \frac{1}{5} \sum_{s=0}^{4} \left[\mathrm{Metric}_{\text{model 1},\, s}(b) - \mathrm{Metric}_{\text{model 2},\, s}(b) \right], \tag{4.7}$$

where s indexes the seeds. Higher values are better for macro F1, micro F1, and PR-AUC, and lower values are better for Hamming loss. The comparison is made for each pair of aggregation

methods (three pairs), and confirmatory inference is limited to the three macro-F1 comparisons. For each pair, the two-sided bootstrap probability is twice the smaller of the proportions of $\Delta(b)$ values at or below zero and at or above zero, with a plus-one correction, capped at one. The three probabilities are adjusted with the Holm procedure at a family-wise significance level of $\alpha = 0.05$. The other metrics are reported with effect sizes and confidence intervals only.

Scored labels and zero denominators. The scent labels included in macro averages are determined once on the original test split and stay the same across all bootstrap replicates. If a resample contains no positive cells for an included scent label, its F1 and average precision are set to 0.0; the scent label stays in the macro average and the resample is not redrawn. Micro F1, micro AP, and Hamming loss are computed over all eligible cells pooled.

Intervals and effect size. A 95% percentile interval is taken from the 2.5th and 97.5th percentiles of the 10,000 differences. The effect size is the mean paired difference, in the units of the metric. Test blends that share a molecule are related, so these intervals may underestimate the true uncertainty.

Appendix A

Chemical Feature Vectors

The exact composition of the 46-dimensional node feature vector and the 10-dimensional edge feature vector used for the intra-molecular GAT encoder are detailed below.

Chemical Property	**Dimensions**	**Mathematical / One-Hot Representation & Details**
Atomic Identity	9	One-hot encoding of key organic and halogen elements in flavor/fragrance compounds: [C, N, O, S, F, Cl, Br, I, Other].
Formal Charge	1	Scalar representation of the localized net electrical charge on the atom.
Atomic Degree	6	One-hot representation of the number of connected covalent bonds to heavy atoms: [0, 1, 2, 3, 4, 5+].
Explicit Valence	7	One-hot encoding of the total valence electrons committed to bonding in the molecule: [0, 1, 2, 3, 4, 5, 6+].
Hybridization State	6	One-hot representation of the orbital configuration: [sp, sp^2, sp^3, sp^3d, sp^3d^2, Other].
Aromaticity	1	Binary flag indicating if the atom is part of a resonance-stabilized aromatic ring structure.
Attached Hydrogens	5	One-hot encoding of the number of hydrogens bound to this heavy atom: [0, 1, 2, 3, 4+].
Ring Size Participation	6	Binary flags representing specific ring sizes to map cyclic functional groups: [3-ring, 4-ring, 5-ring, 6-ring, 7-ring, 8-ring].
Ring Membership	1	Binary flag indicating whether the atom is enclosed within any ring structure.
Chirality Tag	4	One-hot representation of tetrahedral stereocenter configuration: [Unspecified, Clockwise (CW), Counterclockwise (CCW), Other].

Table A.1: Specification of the 46-Dimensional Atom (Node) Feature Vector (X_i).

Chemical Property	**Dimensions**	**Mathematical / One-Hot Representation & Details**
Bond Type	4	One-hot representation of the bond's electronic sharing structure: [Single, Double, Triple, Aromatic].
Conjugation	1	Binary flag indicating if the bond belongs to a continuous, overlapping pi-electron system.
Ring Membership	1	Binary flag indicating if the bond is closed within a ring structure.
Stereochemical Config	4	One-hot representation of spatial configuration around double bonds: [None, Stereo-Z, Stereo-E, Other].

Table A.2: Specification of the 10-Dimensional Bond (Edge) Feature Vector (E_{ij}).

A.1 Symmetry Collapse: Representation-Theoretic Derivation

This appendix provides the fuller representation-theoretic argument for a possible expressivity limitation of linear, order-independent readouts at the mixture level, summarised in the mean-pooling discussion of the main text.

Consider a molecular graph G whose set of node embeddings M occupies a d-dimensional representation space $\mathcal{U}$. The structural symmetries of the graph are captured by its automorphism group $\mathcal{H}$, which collects all permutations of the node indices that leave the graph structure unchanged. When a linear, $\mathcal{H}$-invariant readout function f is applied to aggregate these embeddings, representation theory requires that it factor through the group-averaging Reynolds operator p_{avg}, defined as [37]:

$$p_{\text{avg}} = \frac{1}{|\mathcal{H}|} \sum_{h \in \mathcal{H}} \rho(h). \tag{A.1}$$

The Reynolds operator projects the full representation space $\mathcal{U}$ onto its trivial, symmetry-fixed subspace $\mathcal{U}_1$. As a consequence, any linear $\mathcal{H}$-invariant map decomposes into a downstream linear map composed with this group-averaging projection [37]:

$$f = f' \circ p_{\text{avg}}. \tag{A.2}$$

This factorization establishes that a linear readout can only operate on the information that survives in $\mathcal{U}_1$. The non-trivial, symmetry-sensitive components $\mathcal{U}_2 \oplus \cdots \oplus \mathcal{U}_m$, which encode the node-to-node variation responsible for structural differences between molecules, can be discarded by the projection [37]. Thus, two distinct combinations that share the same element-wise sum or mean can become indistinguishable to a downstream classifier. This is a possible information loss in the representation; it does not show that such collisions dominate the present dataset. The result therefore motivates testing attention-based aggregation alongside mean pooling, not replacing mean pooling or assuming that attention will perform better [33, 37].

A.2 Dataset Profiles and Canonical Vocabulary

A.2.1 Single-Molecule Harmonized Dataset Sample

Table A.3 provides representative single-molecule entries resulting from the consolidation of the GoodScents and Leffingwell databases under RDKit organic sanitization and canonicalization.

Table A.3: Representative sample of harmonized single odorants.

CID	Common Name	Canonical SMILES	Harmonized Descriptors
1183	Vanillin	`COc1cc(C=O)ccc1O`	chocolate, creamy, sweet, vanilla
1254	Menthol	`CC1CCC(C(C)C)C(O)C1`	cooling, mentholic, minty, woody
3314	Eugenol	`C=CCc1ccc(O)c(OC)c1`	cinnamon, clove, phenolic, spicy, woody
10582	Myrtenol	`CC1(C)C2CC=C(CO)C1C2`	balsamic, camphoreous, herbal, pine
31276	Isoamyl acetate	`CC(=O)OCCC(C)C`	banana, estery, fruity, pear, sweet
87630	2-Decen-1-ol	`CCCCCCCC=CCO`	citrus, fatty, floral, rose, waxy
637511	Cinnamaldehyde	`O=C/C=C/c1ccccc1`	candy, cinnamon, spicy, warm

A.2.2 Pairwise Blend Mixture Sample

Table A.4 presents sample binary mixtures from the pairwise blend benchmark dataset compiled by Sisson et al. [33], illustrating the constituent molecules and their catalog-recorded blend descriptors.

Table A.4: Sample pairwise odor mixtures from the Sisson et al. benchmark with catalog-recorded descriptors.

Blend	Component 1 (CID)	Component 2 (CID)	Catalog-Recorded Descriptors
B-01	Acetaldehyde (177)	2-Heptanol (10976)	citrus, fruity, pungent
B-02	Acetoin (179)	Bornylcyclohexanol (111394)	buttery, creamy, woody
B-03	2-Oxobutanoic acid (58)	Methyl dihydrojasmonate (102861)	brown, caramellic, floral
B-04	2-Oxobutanoic acid (58)	γ-Undecalactone (7714)	brown, caramellic, fruity
B-05	Pentopyranose (229)	Perillaldehyde (16441)	herbal, spicy
B-06	2-Oxobutanoic acid (58)	Butyric acid (264)	cheesy, sour
B-07	Cinnamaldehyde (637511)	Eugenol (3314)	spicy

A.2.3 Standardized 104-Odor Descriptor Vocabulary Gate

Table A.5 enumerates the complete 104-descriptor standardized vocabulary gate enforced during Phase 1 preprocessing.

Table A.5: The 104 standardized odor descriptors (alphabetical).

1. acetic	27. clean	53. honey	79. potato
2. acidic	28. cocoa	54. jammy	80. powdery
3. alcoholic	29. coconut	55. juicy	81. pungent
4. aldehydic	30. coffee	56. lactonic	82. roasted
5. alliaceous	31. cooling	57. leathery	83. rooty
6. amber	32. corn	58. licorice	84. rummy
7. ammoniacal	33. coumarinic	59. malty	85. salty
8. animal	34. creamy	60. marine	86. smoky
9. anise	35. dairy	61. meaty	87. soapy
10. aromatic	36. dusty	62. medicinal	88. solvent
11. balsamic	37. earthy	63. melon	89. sour
12. berry	38. eggy	64. mentholic	90. spicy
13. bitter	39. estery	65. minty	91. sulfurous
14. bready	40. ethereal	66. moldy	92. sweet
15. brown	41. fatty	67. mossy	93. tarragon
16. burnt	42. fermented	68. mushroom	94. thujonic
17. buttery	43. fishy	69. musk	95. toasted
18. cabbage	44. floral	70. mustard	96. tobacco
19. camphoreous	45. fresh	71. musty	97. tomato
20. caramellic	46. fruity	72. nutty	98. tonka
21. celery	47. fungal	73. oily	99. tropical
22. cheesy	48. fusel	74. onion	100. vanilla
23. chemical	49. garlic	75. orris	101. vegetable
24. cherry	50. green	76. peach	102. waxy
25. chocolate	51. hay	77. phenolic	103. winey
26. citrus	52. herbal	78. pine	104. woody

A.3 Project Timeline and Gantt Charts

This appendix documents the research and manuscript development schedule across two sequential phases, separating completed work leading up to Graded Defense 1 from the remaining implementation, evaluation, and final defense milestones.

A.3.1 Topic to Graded Defense 1 (August – November 2026)

Figure A.1 summarizes the initial execution timeline across Stages 1 through 5. It spans topic ideation, literature sourcing, and Chapters 1–4 drafting (Stages 1–2); post-proposal revisions, chemistry consultation, and the topic redefense (Stage 3); data consolidation, reliability eligibility masking, and component-disjoint graph carving (Stage 4); and interactive prototype development and Graded Defense 1 preparation (Stage 5), interspersed with scheduled adviser reviews and formal defense milestones.

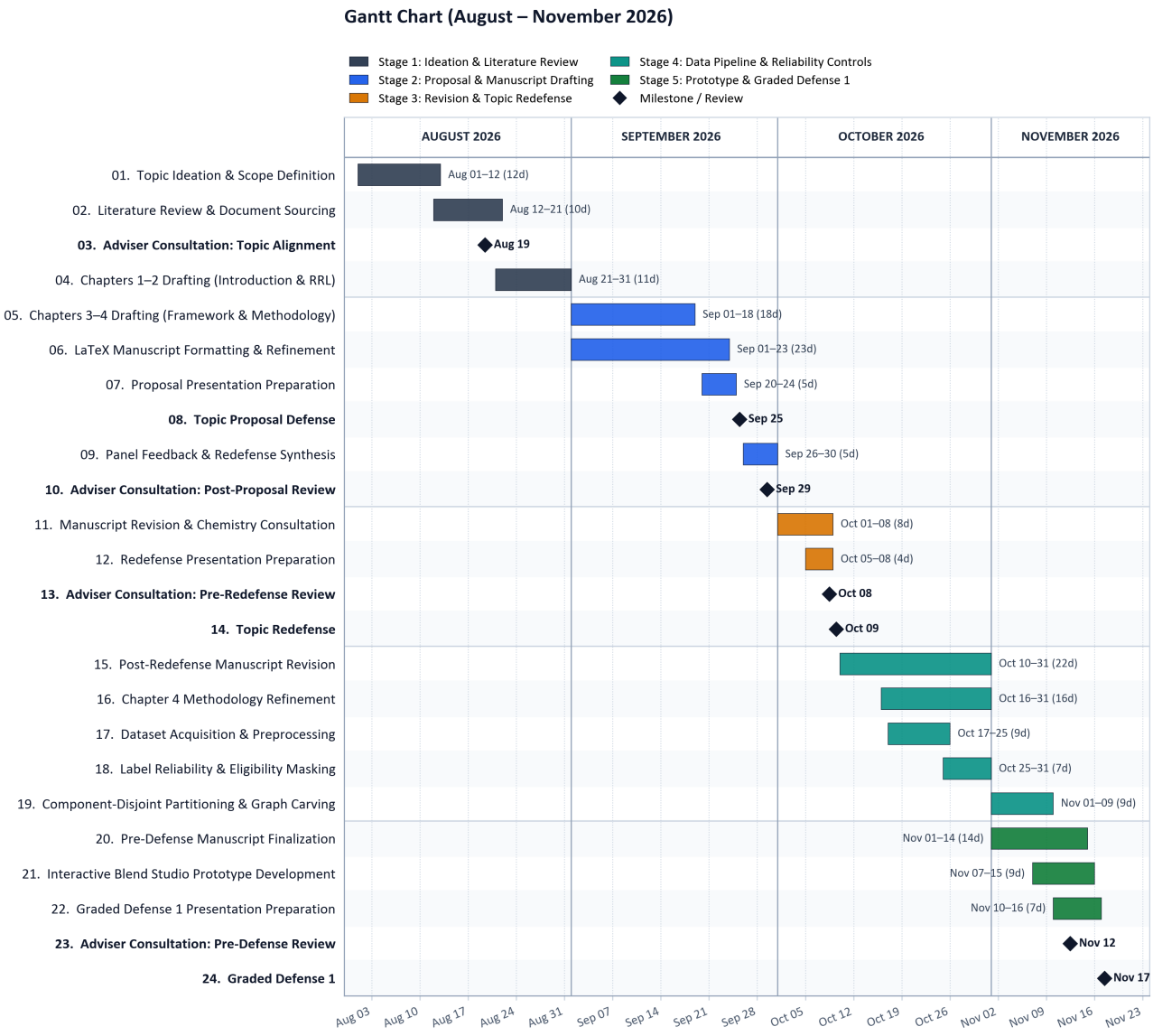


Figure A.1: Topic to Graded Defense 1 Gantt Chart (August – November 2026).

A.3.2 Post-Graded Defense 1 to Final Defense (November 18, 2026 – February 2027)

Figure A.2 outlines the remaining schedule across Stages 6 through 9 following Graded Defense 1. It covers panel revisions and shared molecular graph encoder implementation (Stage 6); matched multi-seed training of the Mean Pooling, Order-Averaged Concatenation, and Set Transformer heads (Stage 7); primary comparative experiments, overlap stratification, controlled label-corruption sweeps, and masked statistical evaluation for RQ1–RQ3 (Stage 8); and Chapters 4–5 drafting, final prototype integration, and manuscript proofreading (Stage 9). Weekly adviser check-ins are scheduled from January through February 2027, while the Preliminary Defense and Final Thesis Defense remain marked as schedule to be determined (TBD).

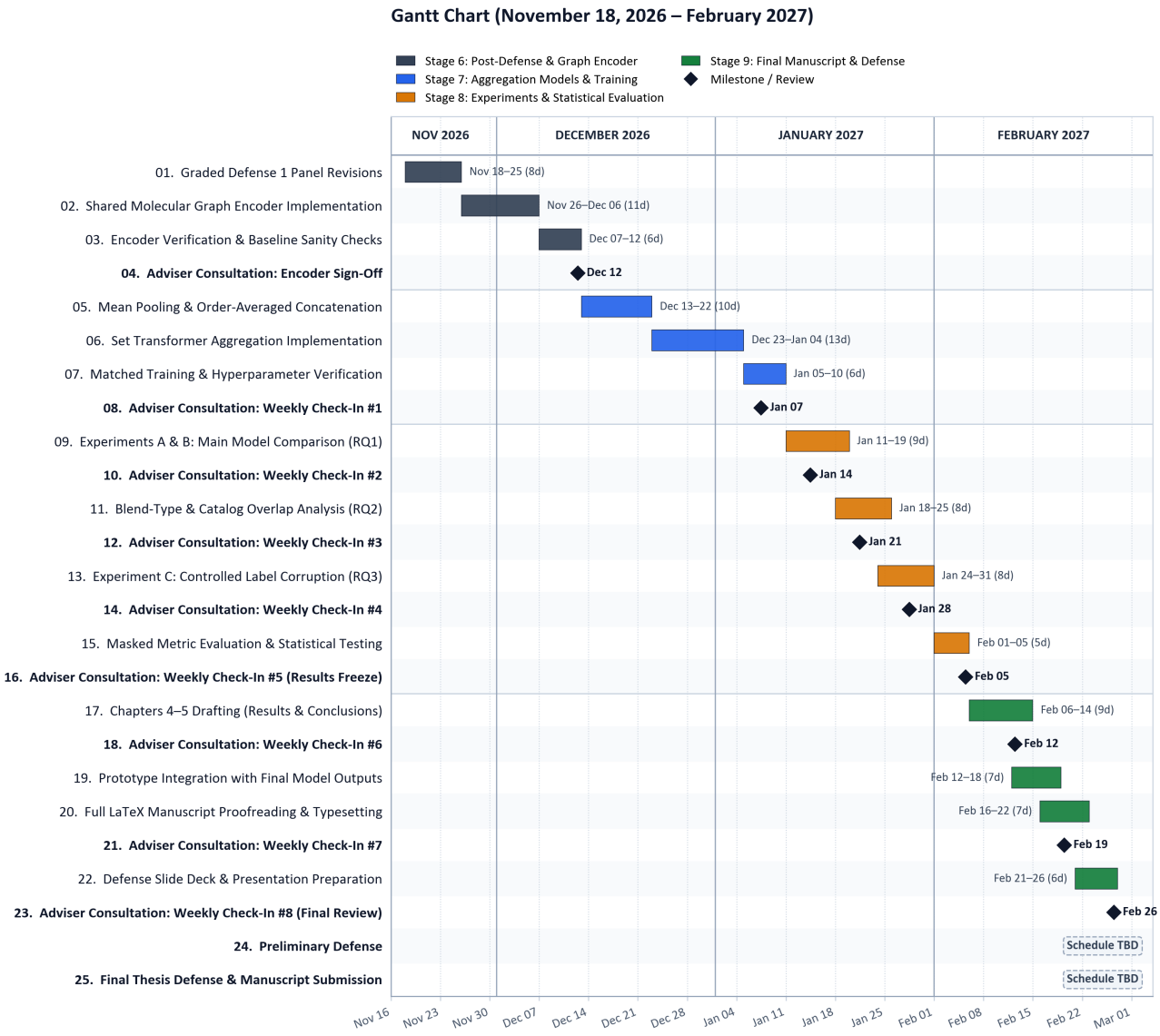


Figure A.2: Post-Graded Defense 1 to Final Defense Gantt Chart (November 18, 2026 – February 2027).

Appendix B

Prototype Application

The purpose and features of this study's prototype application are detailed below.

B.1 Purpose and Status

The prototype application is an Unordered-Pair Blend Scent Predictor. Its main feature is that it enables the user to pick two molecules and see the predicted scent labels that arise from the three different aggregation methods compared in this study. Its purpose is to demonstrate the study's pipeline and make it interactive. However, as it is only a prototype, the scores and numbers shown are for illustrative purposes only and are not the real outputs of the final saved models.

B.2 Main Features

B.2.1 Home Page

The landing page shows a brief overview of the thesis and its main purpose. It also shows key numbers, such as the number of molecules available in the dictionary, the total number of scent labels, and the number of models that can be used to predict the scent labels (Figure B.1).

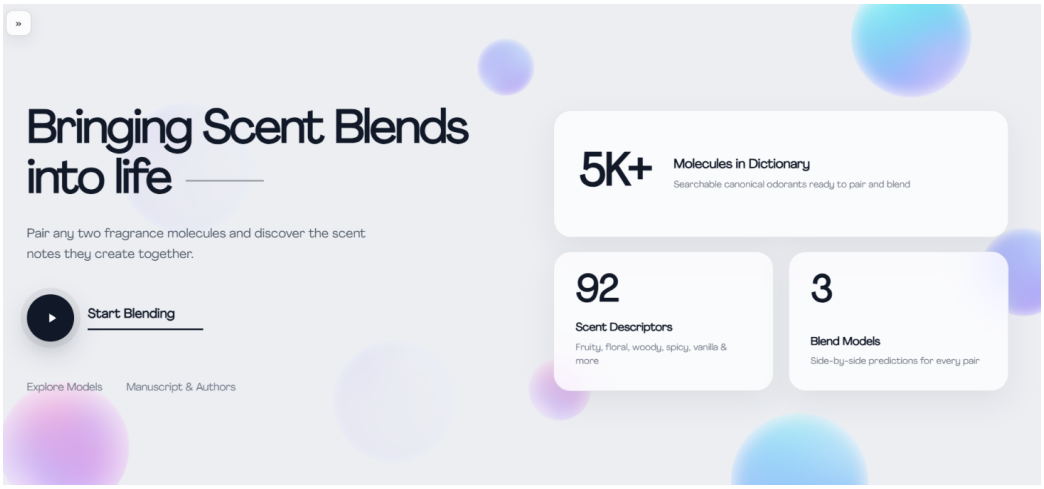


Figure B.1: Home page of the prototype application.

B.2.2 Blend Studio

The Blend Studio page contains the main feature of the prototype, which allows the user to select the two molecules to be blended together. There are two main boxes, one for Molecule A and one for Molecule B, which are the components to be combined. Each box also shows the molecule's SMILES and a visual of its chemical structure for illustrative purposes only. The user can select a blend component by searching by name, CID, or SMILES, or by browsing the dictionary, which is further explained in Section B.2.4. This page also has a swap button for Molecule A and Molecule B, which showcases the concept of input-order invariance. To combine the two molecules for scent label prediction, the user clicks the "Blend this pair" button (Figure B.2).

B.2.3 Results by Aggregation Method

After the user clicks the "Blend this pair" button, the results appear following a short loading screen. The results card features three tabs, one for each method: Mean Pooling, Order-Averaged Concatenation, and Set Transformer. Each tab lists the top five scent labels with the highest probability scores and their source (which molecule the label comes from). A drop-down button also lists all 92 scent labels with their corresponding probability scores, sources, and cutoff status for each method (Figure B.3).

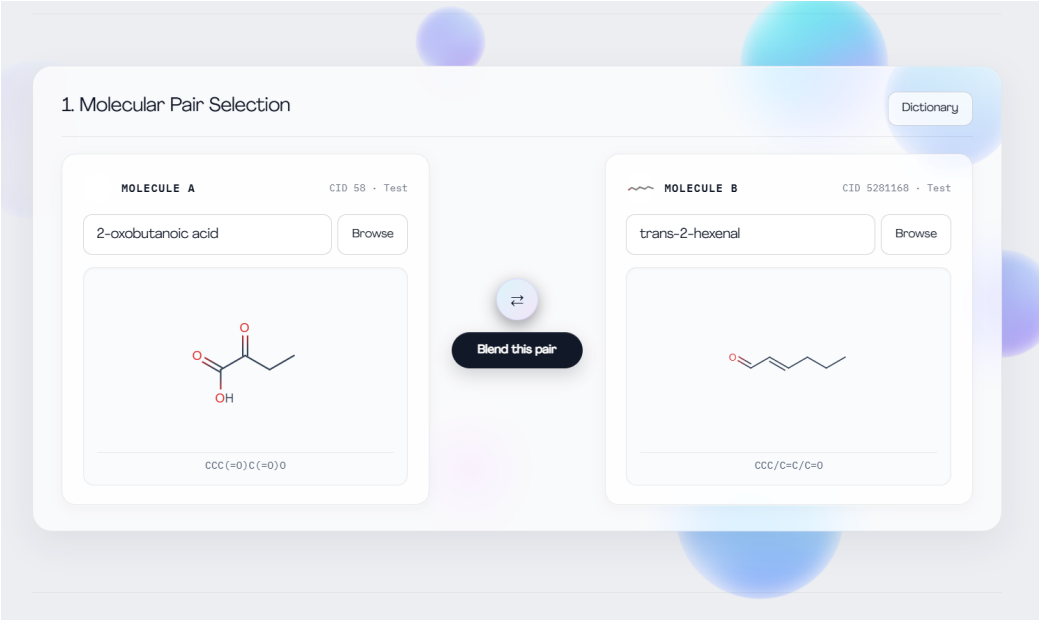


Figure B.2: Blend Studio with Molecule A and Molecule B selected.

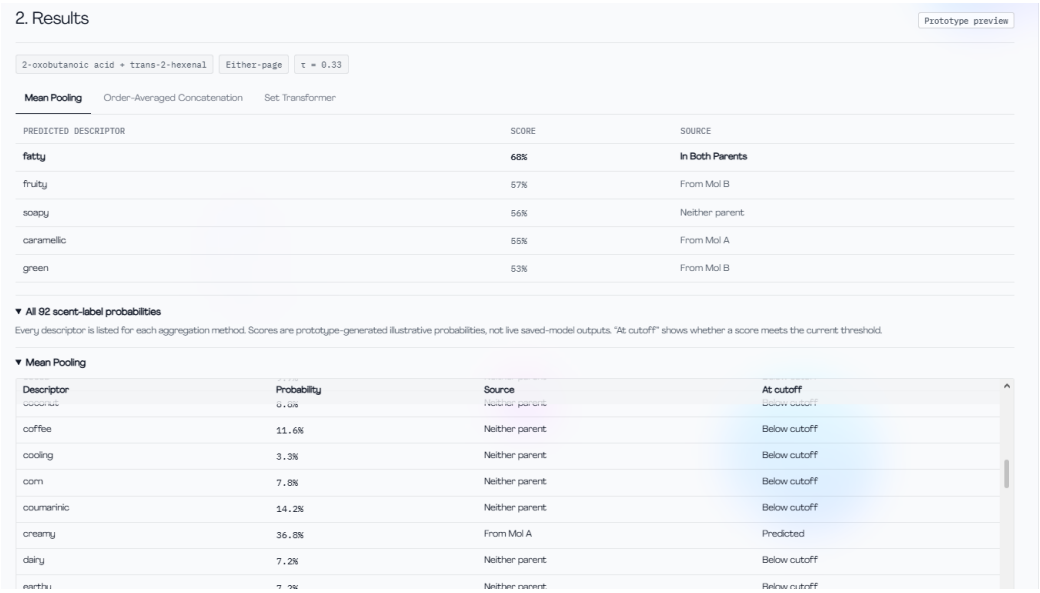


Figure B.3: Results card showing the three aggregation method tabs.

B.2.4 About the Study and Molecule Dictionary

The About page explains the three methods and the research questions. The Dictionary lists the 5,712 molecules and can be filtered by train, validation, or test partition. Each molecule listing shows its CID, name, the cluster partition it belongs to, molecular weight, chemical structure, and SMILES, as well as buttons that allow it to be loaded into either the Molecule A or Molecule B box in the Blend Studio (Figures B.4 and B.5).

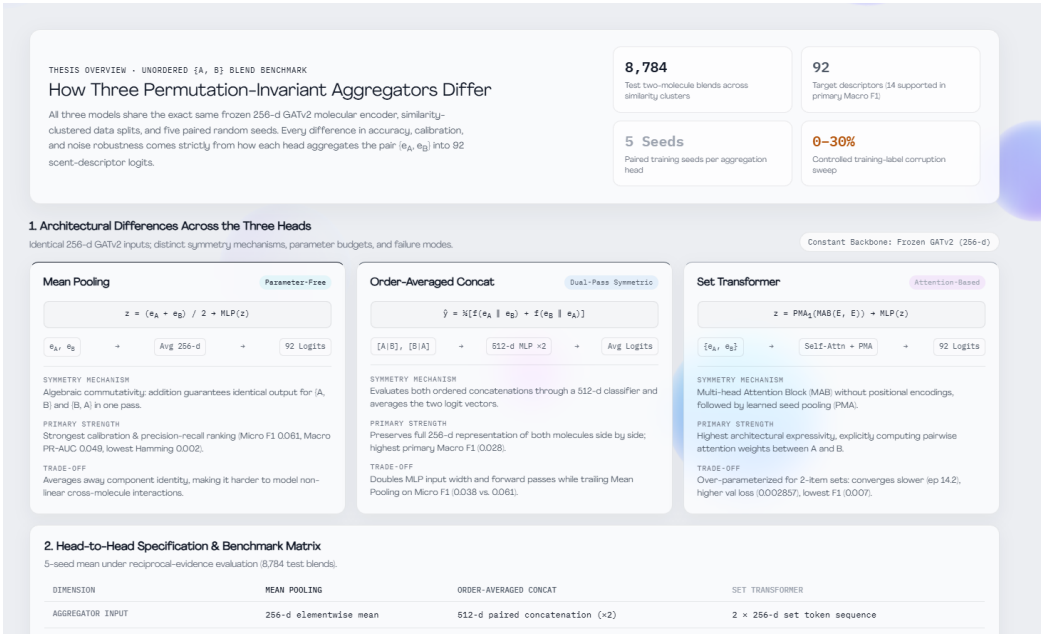


Figure B.4: About page of the prototype application.

B.3 Basic Usage Flow

1. Choose Molecule A and Molecule B.
2. Click “Blend this pair”.
3. Compare the three method tabs.

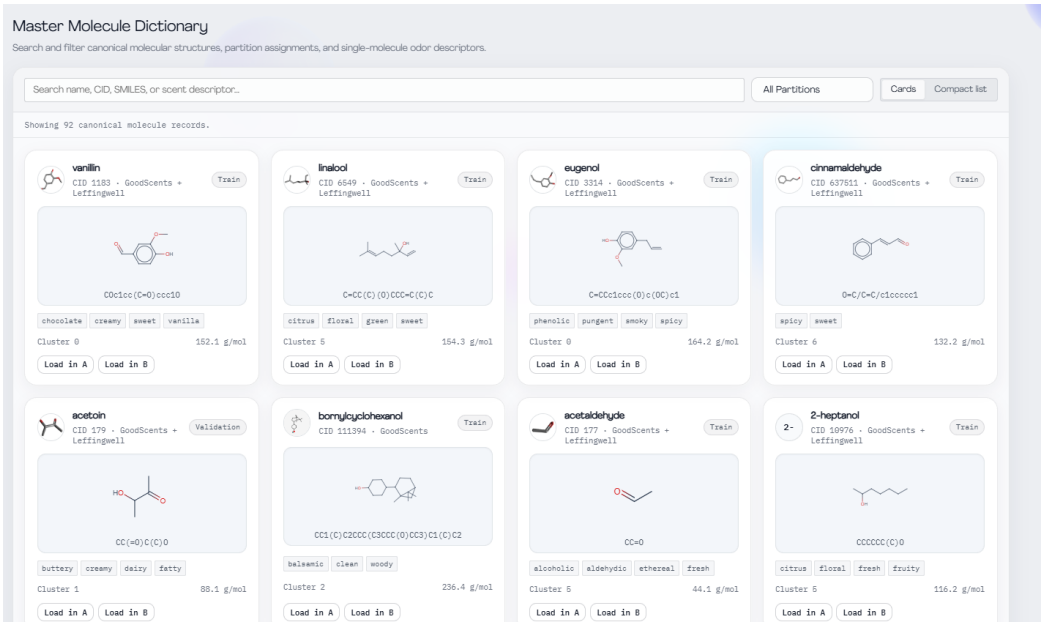


Figure B.5: Molecule Dictionary with partition filters.

B.4 Limitations of the Prototype

The prototype only allows the prediction of scent labels from combining two molecules. The scores that appear are illustrative and do not show the output of the real saved trained models.

REFERENCES

[1] R. Achebouche, A. Tromelin, K. Audouze, and O. Taboureau, *Application of artificial intelligence to decode the relationships between smell, olfactory receptors and small molecules*, Scientific Reports, 12 (2022), p. 18817.

[2] R. L. Almeida, V. G. Maltarollo, and F. G. F. Coelho, *Overcoming class imbalance in drug discovery problems: Graph neural networks and balancing approaches*, Journal of Molecular Graphics and Modelling, 126 (2024), p. 108627.

[3] D. Ameta, S. Kumar, R. Mishra, L. Behera, A. Chakraborty, and T. Sandhan, *Odor classification: Exploring feature performance and imbalanced data learning techniques*, PLOS ONE, 20 (2025), p. e0322514.

[4] A. R. N. Aouichaoui, F. Fan, S. S. Mansouri, J. Abildskov, and G. Sin, *Combining group-contribution concept and graph neural networks toward interpretable molecular property models*, Journal of Chemical Information and Modeling, 63 (2023), pp. 725–744.

[5] Y. Bestgen, *Please, don't forget the difference and the confidence interval when seeking for the state-of-the-art status*, in Proceedings of the Thirteenth Language Resources and Evaluation Conference, Marseille, France, June 2022, European Language Resources Association, pp. 5956–5962.

[6] A. L. Bierling, A. Croy, T. Jesgarzewsky, M. Rommel, G. Cuniberti, T. Hummel, and I. Croy, *A dataset of laymen olfactory perception for 74 mono-molecular odors*, Scientific Data, 12 (2025), p. 347.

[7] J. B. Castro, T. J. Gould, R. Pellegrino, Z. Liang, L. A. Coleman, F. Patel, D. S. Wallace, T. Bhatnagar, J. D. Mainland, and R. C. Gerkin, *Pyrfume: A window to the world's olfactory data*, bioRxiv, (2022).

[8] S. Chithrananda, J. Amores, and K. K. Yang, *Mapping the combinatorial coding between olfactory receptors and perception with deep learning*, bioRxiv, (2026).

[9] G. Coureaud, T. Thomas-Danguin, J.-C. Sandoz, and D. A. Wilson, *Biological constraints on configural odour mixture perception*, J Exp Biol, 225 (2022).

[10] R. Datta, J. Nistane, A. Sose, H. Sahu, and R. Ramprasad, *Machine learning for green solvents: Assessment, selection and substitution*, Advanced Science, (2025).

[11] M. Fey and J. E. Lenssen, *Fast graph representation learning with pytorch geometric*, CoRR, abs/1903.02428 (2019).

[12] H. Fooladi, T. N. L. Vu, M. Mathea, and J. Kirchmair, *Evaluating machine learning models for molecular property prediction: Performance and robustness on out-of-distribution data*, Journal of Chemical Information and Modeling, 65 (2025), pp. 9871–9891.

[13] Y. Fukutani, M. Abe, H. Saito, R. Eguchi, T. Tazawa, C. A. de March, M. Yohda, and H. Matsunami, *Antagonistic interactions between odorants alter human odor perception*, Current Biology, 33 (2023), pp. 2235–2245.e4. doi: 10.1016/j.cub.2023.04.072.

[14] D. Gasser, S. Singh, J. Freiherr, and A. Grasskamp, *Owsum: algorithmic odor prediction and insight into structure-odor relationships*, Journal of Cheminformatics, 15 (2023).

[15] Q. Guo, S. Hernandez-Hernandez, and P. J. Ballester, *Umap-based clustering split for rigorous evaluation of ai models for virtual screening on cancer cell lines**, Journal of Cheminformatics, 17 (2025), p. 94.

[16] R. Hagan, D. Moore, F. McGlone, and S. C. Walker, *Olfactory scene analysis: Does analytical visual processing predict superior identification of component odours in a complex mixture?*, Perception, 54 (2025), pp. 408–430. PMID: 40165414.

[17] S. Han, H. Fu, Y. Wu, G. Zhao, Z. Song, F. Huang, Z. Zhang, S. Liu, and W. Zhang, *Himgnn: a novel hierarchical molecular graph representation learning framework for property prediction*, Briefings in Bioinformatics, 24 (2023), p. bbad305.

[18] F. Hatami and M. Moradi, *Asymmetric focal loss improves graph neural network prediction of drug-drug interactions*, 2026.

[19] Y. P. Heng, H. Y. Lee, J. W. Chong, R. R. Tan, K. B. Aviso, and N. G. Chemmangattuvalappil, *Incorporating machine learning in computer-aided molecular design for fragrance molecules*, Processes, 10 (2022).

[20] R. Joeres, D. B. Blumenthal, and O. V. Kalinina, *Data splitting to avoid information leakage with datasail*, Nature Communications, 16 (2025), p. 3337.

[21] B. K. Lee, E. J. Mayhew, B. Sanchez-Lengeling, J. N. Wei, W. W. Qian, K. A. Little, M. Andres, B. B. Nguyen, T. Moloy, J. Yasonik, J. K. Parker, R. C. Gerkin, J. D. Mainland, and A. B. Wiltschko, *A principal odor map unifies diverse tasks in olfactory perception*, Science, 381 (2023), pp. 999–1006.

[22] J. Lee, Y. Lee, J. Kim, A. R. Kosiorek, S. Choi, and Y. W. Teh, *Set transformer: A framework for attention-based permutation-invariant neural networks*, 2019.

[23] S. Li, X. Xia, S. Ge, and T. Liu, *Selection-based pseudo-labeling for multi-label learning with noisy labels: Estimating the noise transition*, in Advances in Neural Information Processing Systems, 2022.

[24] A. Paul, *Dextrosinistral reading of smiles notation: Investigation into origin of non-sense code from string manipulations*, Digital Chemical Engineering, 15 (2025), p. 100222.

[25] R. Pellegrino, E. J. Mayhew, J. Margolis, M. Andres, A. B. Wiltschko, R. C. Gerkin, and J. D. Mainland, *Odors smell like their components: A linear framework for predicting olfactory mixture perception*, bioRxiv, (2026).

[26] S. Qin, S. Jiang, J. Li, P. Balaprakash, R. C. Van Lehn, and V. M. Zavala, *Capturing molecular interactions in graph neural networks: a case study in multi-component phase equilibrium*, Digital Discovery, 2 (2023), pp. 138–151.

[27] O. Rainio, J. Teuho, and R. Klén, *Evaluation metrics and statistical tests for machine learning*, Scientific Reports, 14 (2024), p. 6086.

[28] B. C. L. Rodrigues, V. V. Santana, S. Murins, and I. B. R. Nogueira, *Molecule generation and optimization for efficient fragrance creation*, Industrial & Engineering Chemistry Research, 63 (2024), pp. 14480–14494.

[29] K. Saini and V. Ramanathan, *Predicting odor from molecular structure: a multi-label classification approach*, Scientific Reports, 12 (2022), p. 13863.

[30] A. Sajan, S. Sluis, R. Haydarlou, S. Abeln, P. Lisena, R. Troncy, C. Verbeek, I. Leemans, and H. Mouhib, *Hierarchies of smell: Structuring the molecular odor space using semantic taxonomies and machine learning*, Chemical Senses, 51 (2026), p. bjag020.

[31] E. I. Sanchez Medina, S. Linke, M. Stoll, and K. Sundmacher, *Gibbsâ€“helmholtz graph neural network: capturing the temperature dependency of activity coefficients at infinite dilution*, Digital Discovery, 2 (2023), pp. 781–798.

[32] H. Shimakawa, A. Kumada, and M. Sato, *Extrapolative prediction of small-data molecular property using quantum mechanics-assisted machine learning*, npj Computational Materials, 10 (2024), p. 11.

[33] L. Sisson, A. A. Barsainyan, M. Sharma, and R. Kumar, *Deep learning for odor prediction on aroma-chemical blends*, ACS Omega, 10 (2025), pp. 8980–8992.

[34] H. Song, M. Kim, D. Park, Y. Shin, and J.-G. Lee, *Learning from noisy labels with deep neural networks: A survey*, IEEE Transactions on Neural Networks and Learning Systems, 34 (2023), pp. 8135–8153.

[35] S. Steshin, *Lo-hi: Practical ml drug discovery benchmark*, 2023.

[36] J. Suh, Y. Hong, and C. Park, *A comparative study of machine learning models on molecular fingerprints for odor decoding*, Communications Chemistry, 8 (2025), p. 278.

[37] M. Talhi, A. Wolf, and A. Monod, *Breaking symmetry bottlenecks in gnn readouts*, 2026.

[38] A. N. Tarekegn, M. Giacobini, and K. Michalak, *A review of methods for imbalanced multi-label classification*, Pattern Recognition, 118 (2021), p. 107965.

[39] G. Tom, C. T. Ser, E. Rajaonson, S. Lo, H. S. Park, B. Lee, and B. Sanchez-Lengeling, *Does this smell the same? learning representations of olfactory mixtures using inductive biases*, Machine Learning: Science and Technology, 6 (2025).

[40] A. Vanacore and M. S. Pellegrino, *Benchmarking procedures for characterizing the extent of rater agreement: A comparative study*, Quality and Reliability Engineering International, 38 (2022), pp. 1404–1415.

[41] T. Wen, X. Cai, and J. Li, *Graph neural networks vs. traditional qsar: A comprehensive comparison for multi-label molecular odor prediction*, Molecules, 30 (2025).

[42] D. S. Wigh, J. M. Goodman, and A. A. Lapkin, *A review of molecular representation in the age of machine learning*, WIREs Computational Molecular Science, 12 (2022), p. e1603.

[43] Y.-X. Wu, X. Wang, A. Zhang, X. He, and T.-S. Chua, *Discovering invariant rationales for graph neural networks*, 2022.

[44] Z. Wu, J. Wang, H. Du, D. Jiang, Y. Kang, D. Li, P. Pan, Y. Deng, D. Cao, C.-Y. Hsieh, and T. Hou, *Chemistry-intuitive explanation of graph neural networks for molecular property prediction with substructure masking*, Nature Communications, 14 (2023), p. 2585.

[45] H. Xie, J. Sun, Y. Shao, S. Li, S. Hou, Y. Sun, and J. Wang, *Molecular odor prediction based on multi-feature graph attention networks*, 2025.

[46] L. Xu, D.-J. Zou, and S. Firestein, *Odor mixtures: A chord with silent notes*, Frontiers in Ecology and Evolution, Volume 11 - 2023 (2023).

[47] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, and A. Smola, *Deep sets*, 2018.

[48] M. Zhang, Y. Hiki, A. Funahashi, and T. J. Kobayashi, *A deep position-encoding model for predicting olfactory perception from molecular structures and electrostatics*, NPJ systems biology and applications, 10 (2024), p. 76.

[49] W. Zhu, Y. Zhang, D. Zhao, J. Xu, and L. Wang, *Hignn: Hierarchical informative graph neural networks for molecular property prediction equipped with feature-wise attention*, 2022.